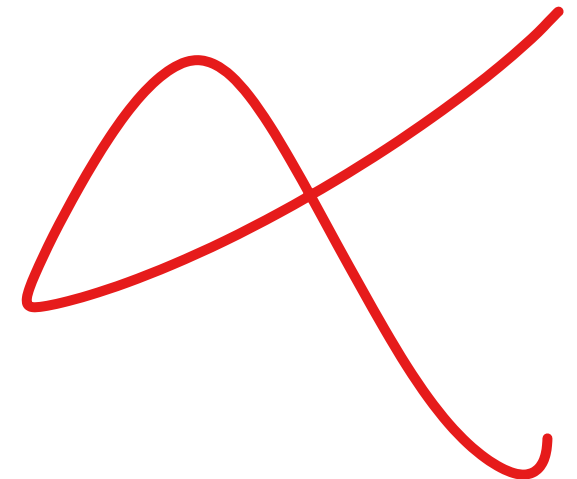
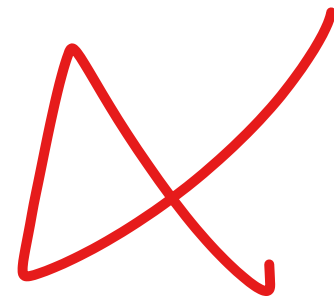


Graph

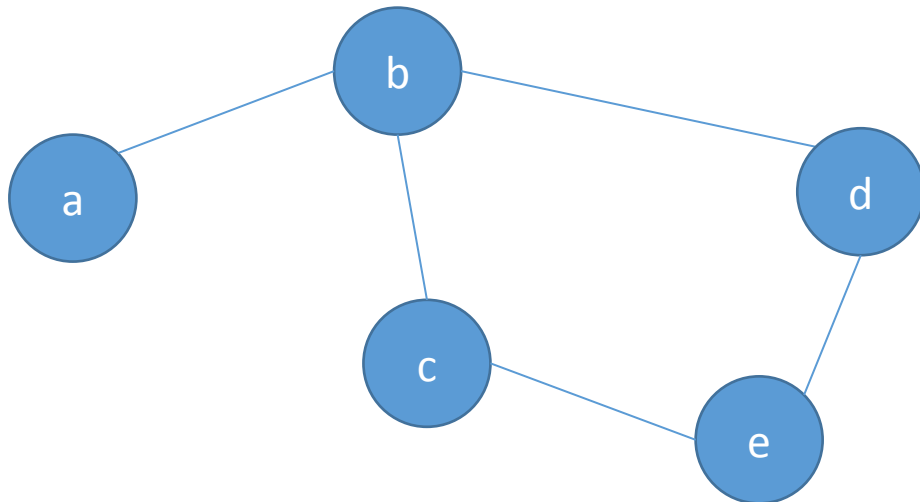


GRAPH TERMINOLOGIES



What is a Graph?

- A graph is a pair of sets (V, E) , where V is a set of nodes, called vertices E is a collection of pairs of vertices, called edges
- $V(G)$ and $E(G)$ represent the sets of vertices and edges of G , respectively.
- We **visualize/draw** the graph and store **represent it mathematically** using the **two sets**.

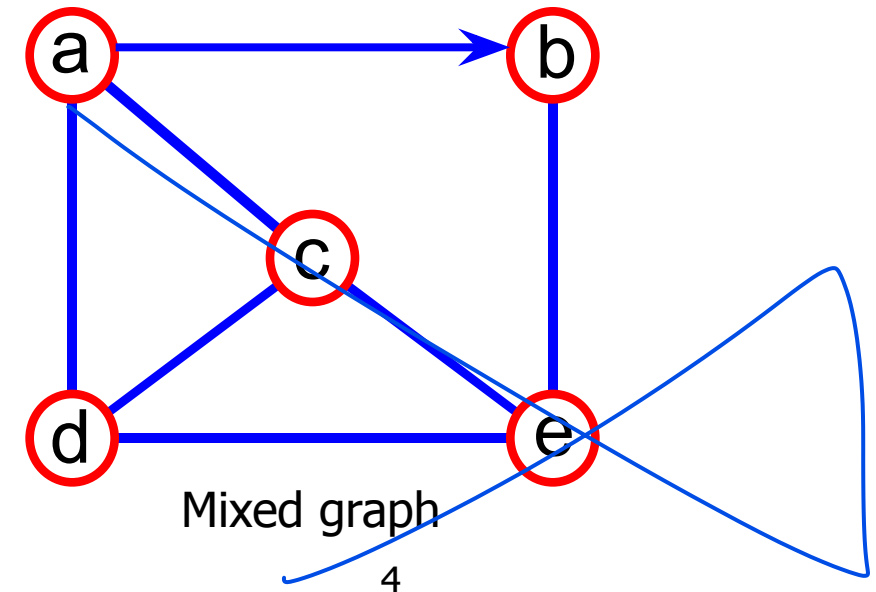


$V = \{a, b, c, d, e\}$

$E = \{(a, b), (b, c), (b, d), (c, e), (d, e)\}$

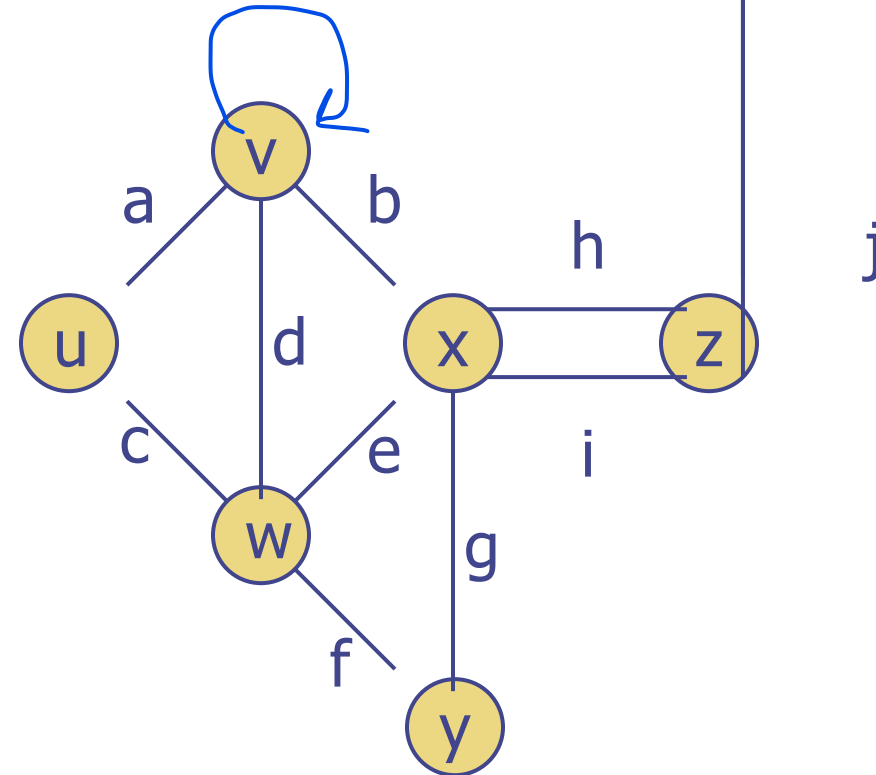
Edge and Graph Types

- **Directed edge**
 - ordered pair of vertices (u,v)
 - first vertex u is the origin
 - second vertex v is the destination
- **Undirected edge**
 - unordered pair of vertices (u,v)
- **Directed graph (Digraph)**
 - all the edges are directed
 - e.g., route network
- **Undirected graph**
 - all the edges are undirected
 - e.g., flight network
- **Mixed graph**
 - some edges are undirected and some edges are directed
 - e.g., a graph modeling a city map



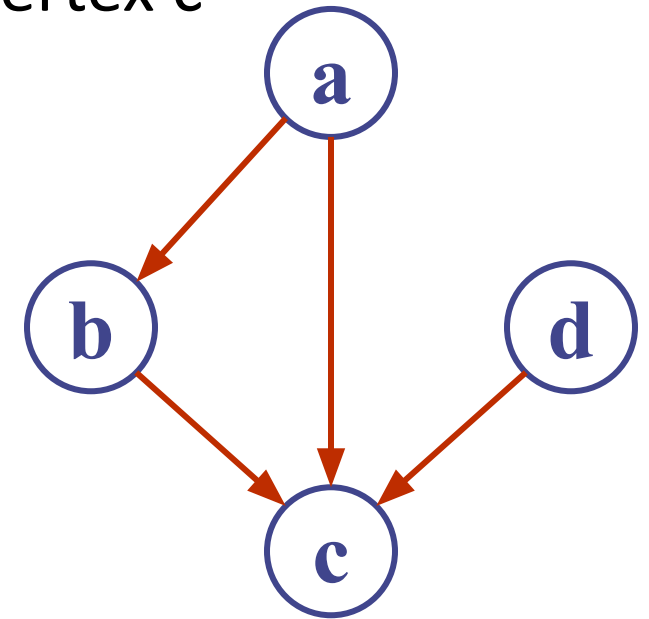
Terminology

- **End vertices (or endpoints) of an edge**
 - u and v are the *endpoints* of a
- **Edges incident to a vertex**
 - a, d, and b are *incident* to v
- **Adjacent vertices**
 - u and v are *adjacent*
- **Degree of a vertex**
 - x has *degree* 5
- **Parallel edges**
 - h and i are *parallel edges*
- **Self-loop**
 - j is a *self-loop*



Terminology (cont.)

- **Outgoing edges of a vertex**
 - (a, b) and (a, c) are outgoing edges of vertex a
- **Incoming edges of a vertex**
 - (b, c) , (d, c) and (a, c) are incoming edges of vertex c
- **In-degree of a vertex**
 - c has *in-degree* 3
 - b has *in-degree* 1
- **Out-degree of a vertex**
 - a has *out-degree* 2
 - b has *out-degree* 1



Terminology (cont.)

- **Path**

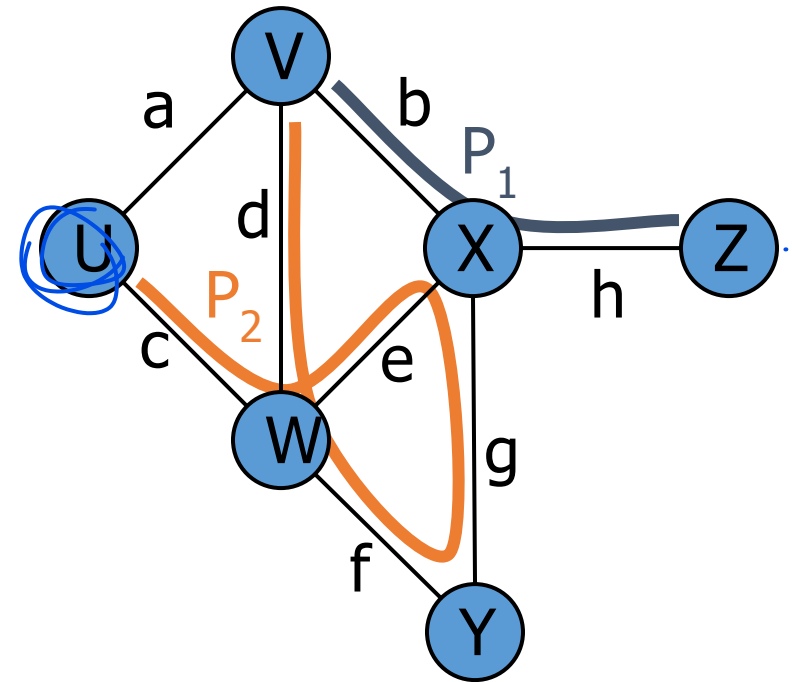
- sequence of alternating vertices and edges
- begins with a vertex
- ends with a vertex
- each edge is preceded and followed by its endpoints

- **Simple path**

- path such that all its **vertices and edges** are **distinct**

- **Examples**

- $P_1 = (V, b, X, h, Z)$ is a simple path
- $P_2 = (U, c, W, e, X, g, Y, f, W, d, V)$ is a path that is not simple



Terminology (Cont.)

- **Cycle**

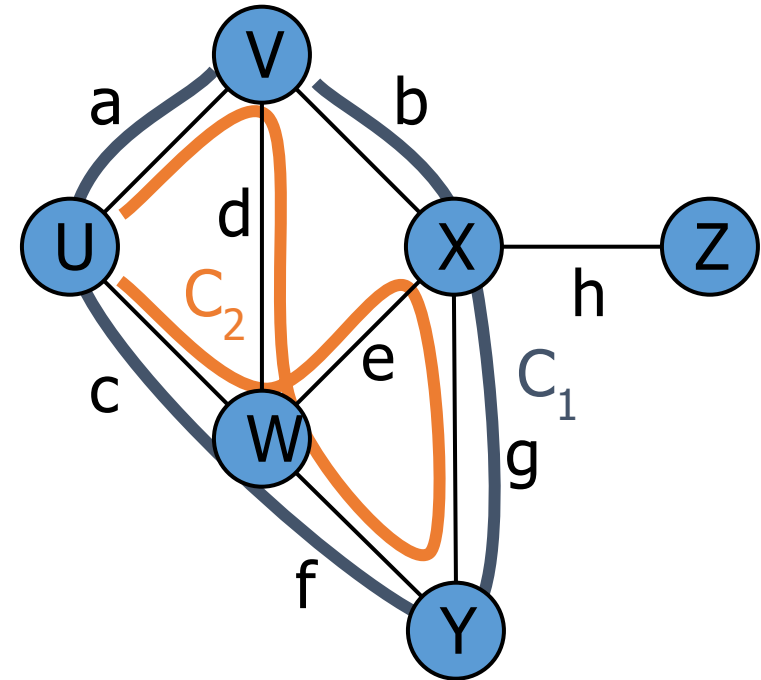
- A cycle is a path whose start and end vertices are the same
- each edge is preceded and followed by its endpoints

- **Simple cycle**

- A cycle is simple if **each edge is distinct** and each vertex is distinct, except for the first and the last one

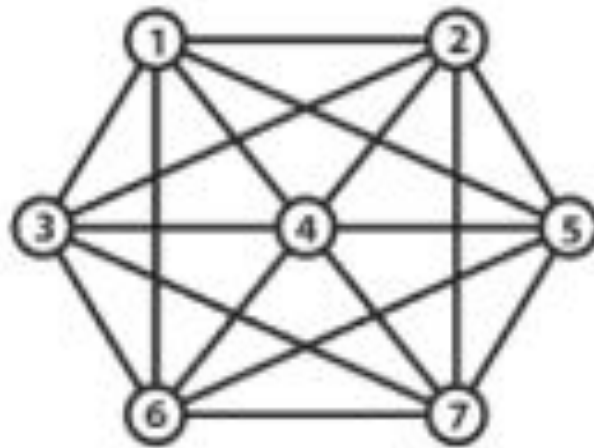
- **Examples**

- $C_1 = (V, b, X, g, Y, f, W, c, U, a, V)$ is a simple cycle
- $C_2 = (U, c, W, e, X, g, Y, f, W, d, V, a, U)$ is a cycle that is not simple

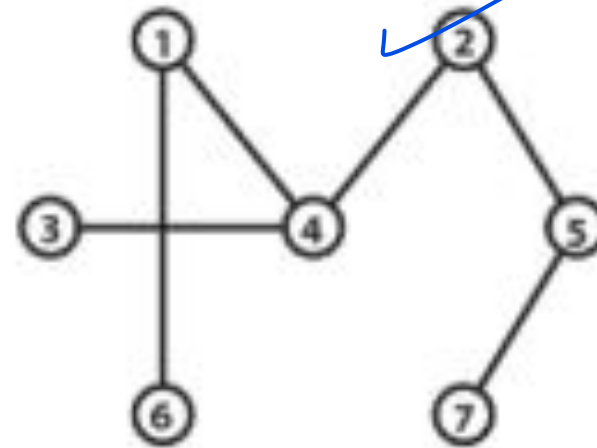


Terminology (Cont.)

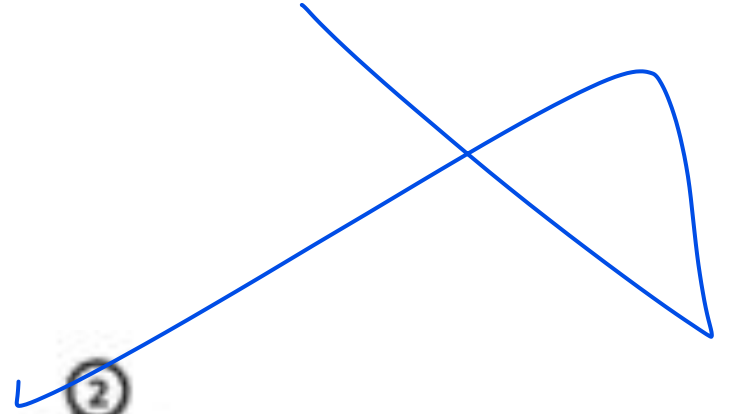
- *Dense* graph: $|E| \approx |V|^2$
- *Sparse* graph: $|E| \approx |V|$



Dense

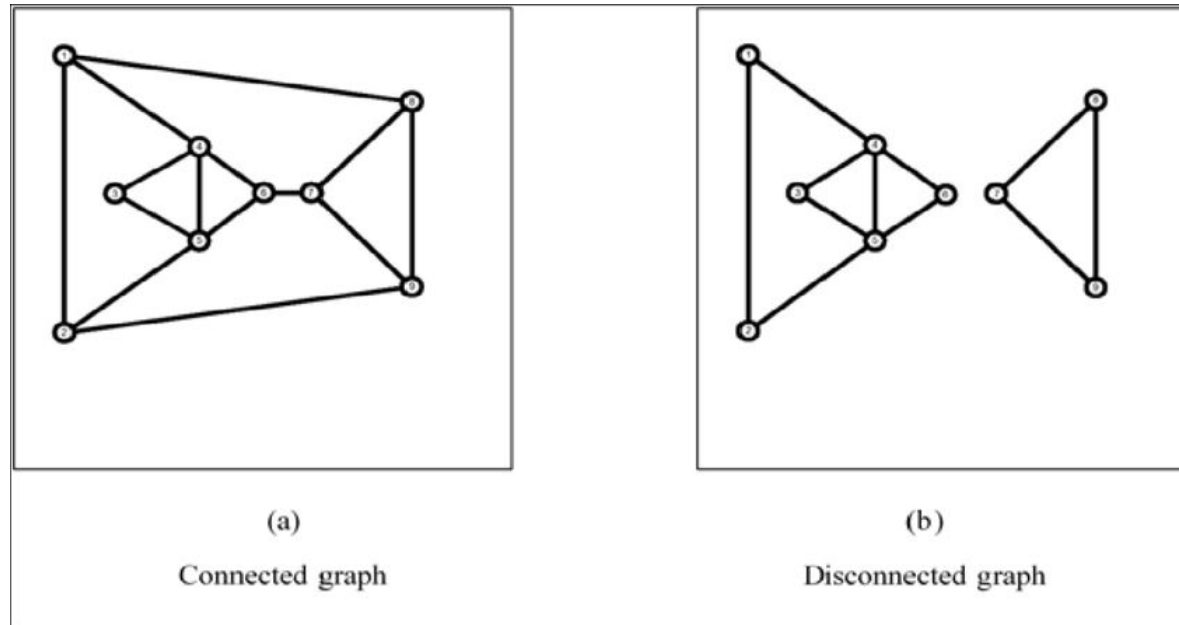


Sparse



Terminology

- In a graph G , two vertices, v_0 and v_1 , are connected if there is a path in G from v_0 to v_1 .
- A graph is connected if, for every pair of distinct vertices v_i and v_j , there is a path from v_i to v_j

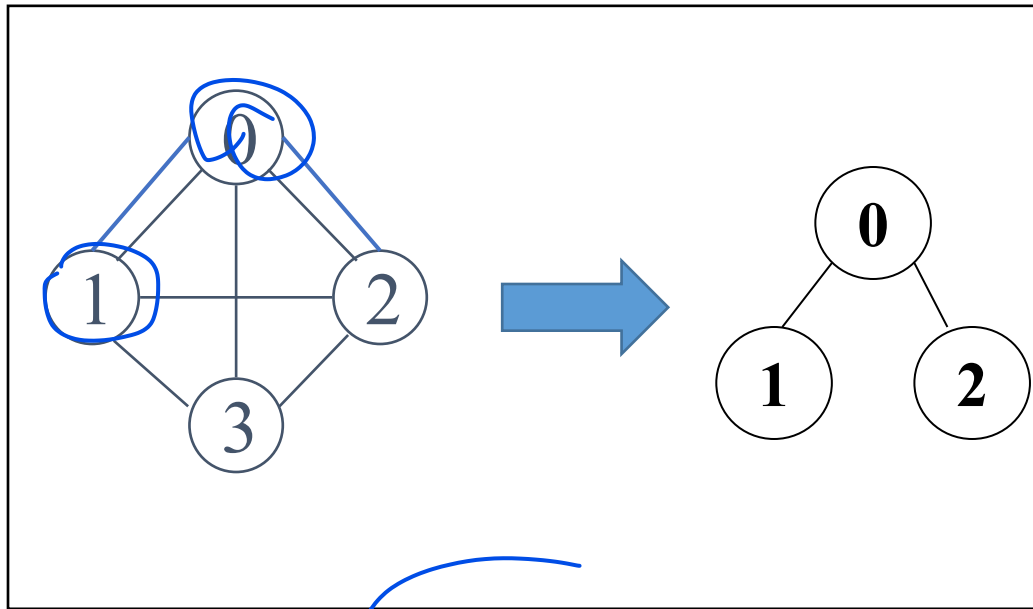


Terminology (Cont.)

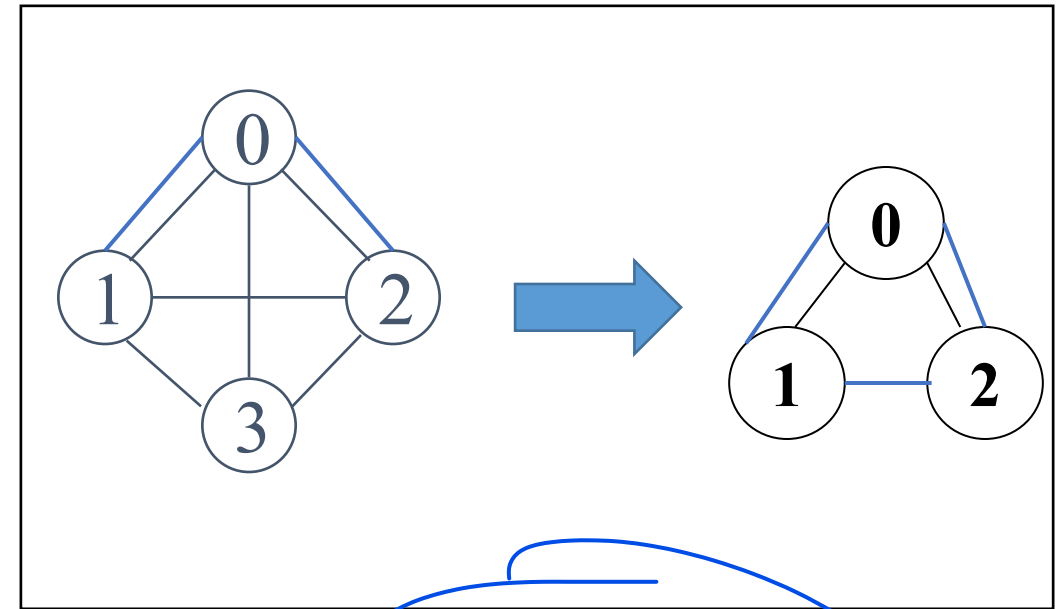
- A ***weighted graph*** associates weights with either the edges or the vertices
- A **complete graph** is a graph that has the maximum number of edges
- In simpler terms, in a complete graph, every node (or point) is directly connected to every other node.
 - for undirected graph with n vertices, the maximum number of edges is $n(n-1)/2$
 - for directed graph with n vertices, the maximum number of edges is $n(n-1)$
 - We can visualize it by drawing small graphs
 - We can prove it using induction.

Terminology (cont.)

- A **subgraph** of G is a graph G' such that G' is a smaller part of the graph G .
- A smaller part of a graph, but if two nodes (vertices) are connected in the original graph, they must be connected in this smaller part too.



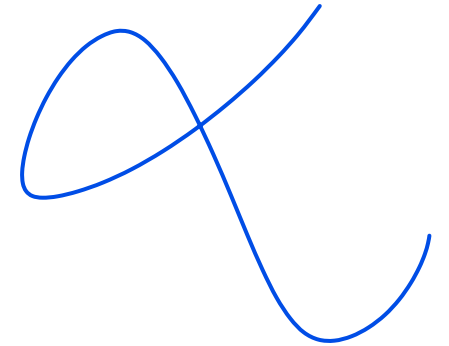
Sub Graph



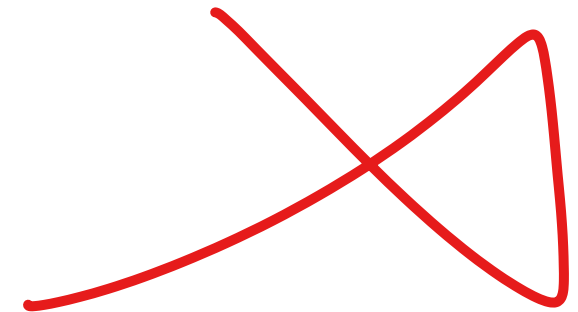
Induced Sub Graph

Terminology (cont.)

- **Spanning Subgraph:** A smaller portion of the original graph, which still includes all the vertices.
- **Tree :** A connected structure without any loop of edges (acyclic).
- **Forest :** A collection of separate trees.
- **Spanning Tree:** A simplified path connecting every graph without any cycles.

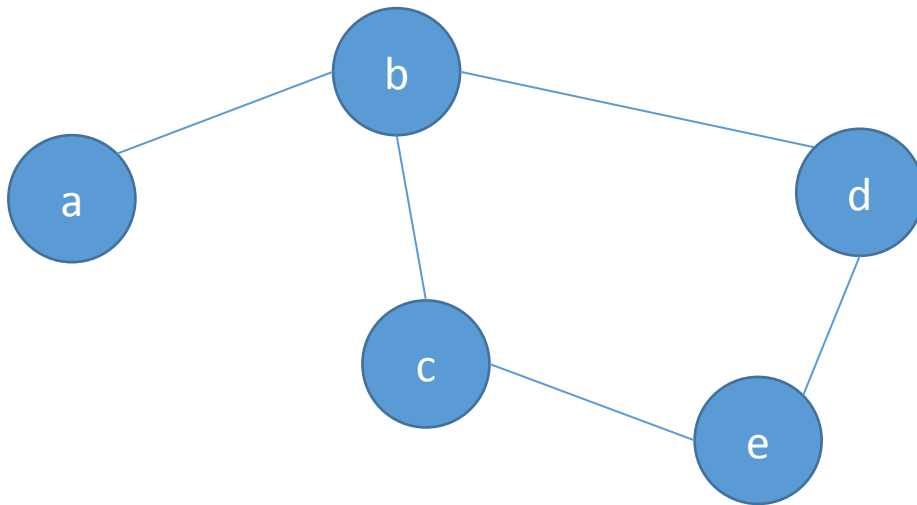


GRAPH REPRESENTATIONS



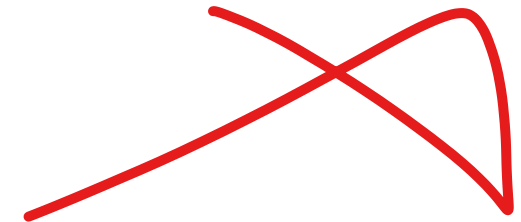
Graph Representation

- We can **visualize/draw** the graph.
- We can represent it MATHEMATICALLY using the **two sets V and E**.



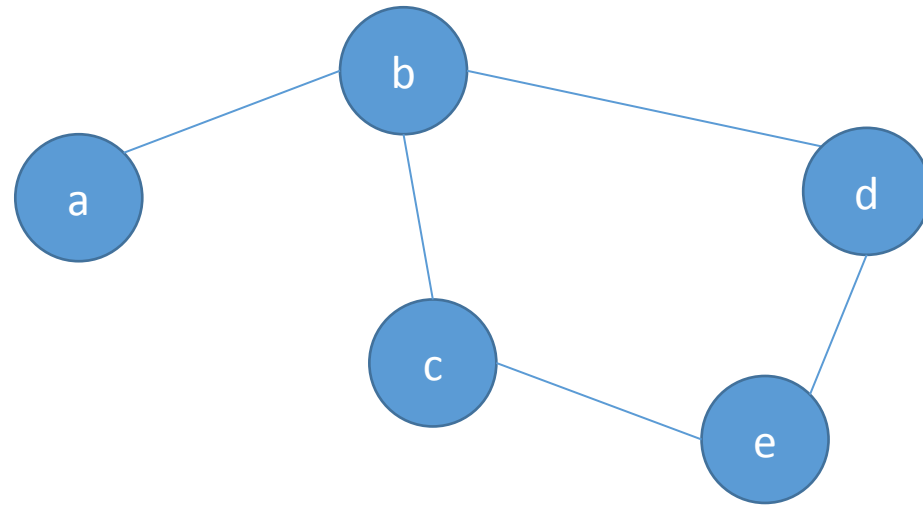
$V = \{a, b, c, d, e\}$

$E = \{(a, b), (b, c), (b, d), (c, e), (d, e)\}$



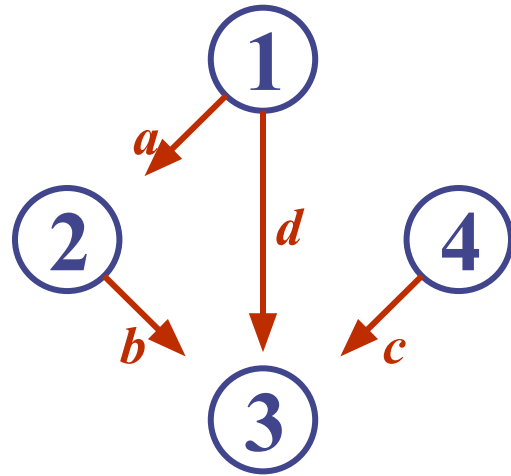
Graph Representation

- We can **visualize/draw** the graph.
- We can represent it in the **COMPUTER MEMORY** using any of the two approaches
 - Adjacency Matrix
 - Adjacency List



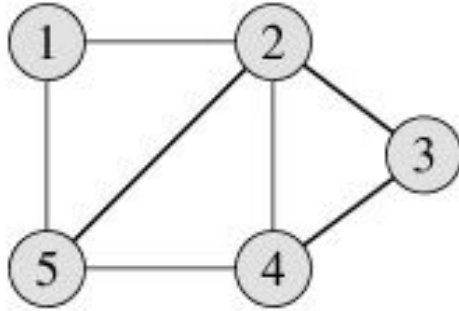
Graph Representation

- Assume $V = \{1, 2, \dots, n\}$
- An *adjacency matrix* represents the graph as a $n \times n$ matrix A :
 - $A[i, j] = 1$ if edge $(i, j) \in E$ (or weight of edge)
= 0 if edge $(i, j) \notin E$



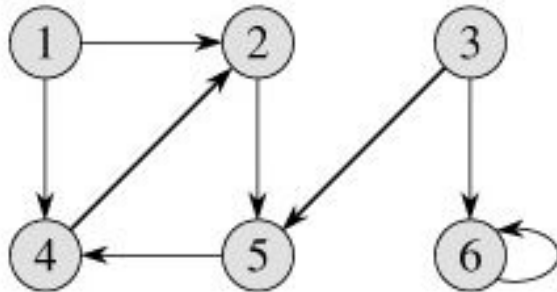
A	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	0	0	0	0
4	0	0	1	0

Adjacency Matrix Representation



Undirected Graph

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0



Directed Graph

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

The adjacency matrix for an undirected graph is symmetric; i.e. : $A[i][j]=A[j][i]$

The adjacency matrix for a digraph need not be symmetric

Adjacency Matrix Representation

- **Pros:**

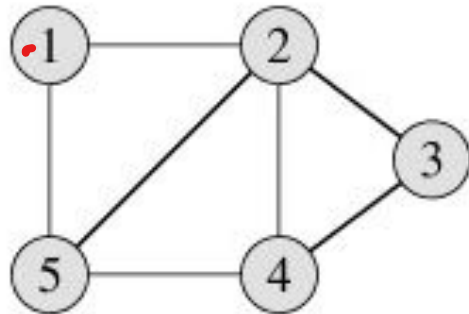
- **Simple** to implement
- Easy and fast to tell if a pair (i, j) is an edge: **simply check if $A[i, j]$ is 1 or 0**
- Can be very **efficient for small graphs**

- **Cons:**

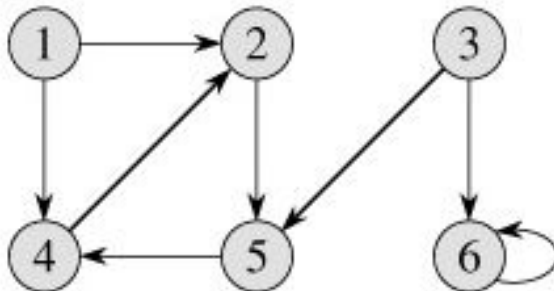
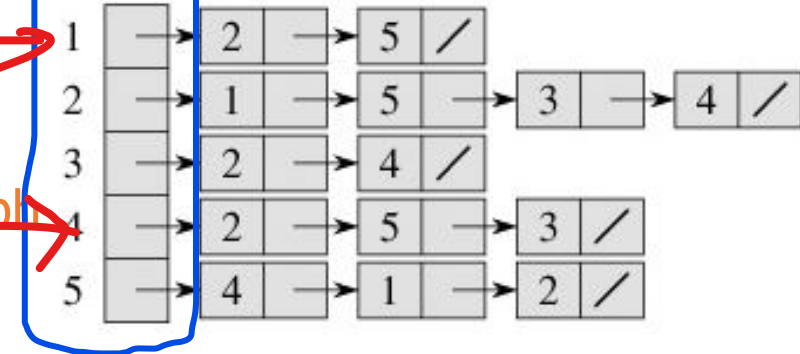
- No matter how few edges the graph has, the **matrix takes $O(n^2)$ in memory**

Adjacency Lists Representation

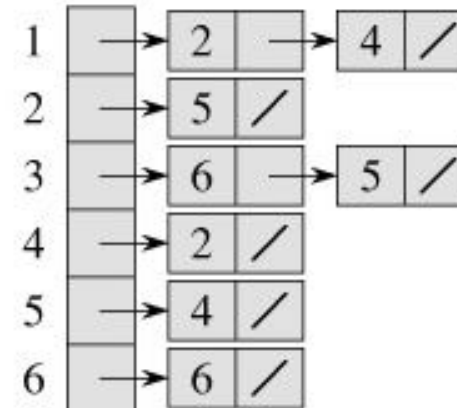
- A graph is represented by a **one-dimensional array L of linked lists**, where
 - L is an array of pointers
 - L[i] is the linked list containing all the nodes adjacent to node i.



Undirected Graph

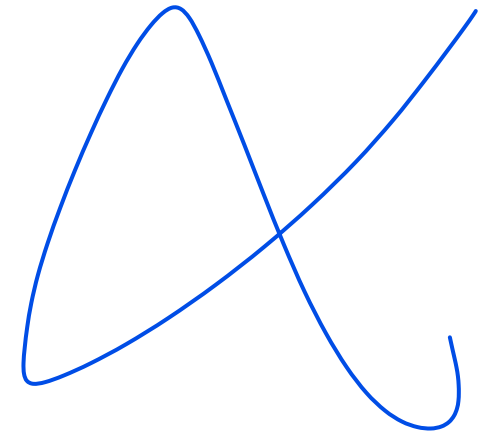


Directed Graph



Adjacency Lists Representation

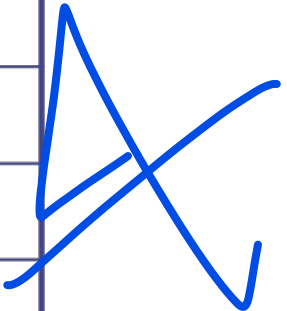
- Pros:
 - Saves on **space (memory)**: the representation takes $O(|V| + |E|)$ memory.
 - Good for large graphs
- Cons:
 - It can take up to **$O(n)$** time to determine if a pair of nodes **(i, j)** is an edge: one would have to search the linked list $L[i]$, which takes time proportional to the length of $L[i]$.



Asymptotic Performance

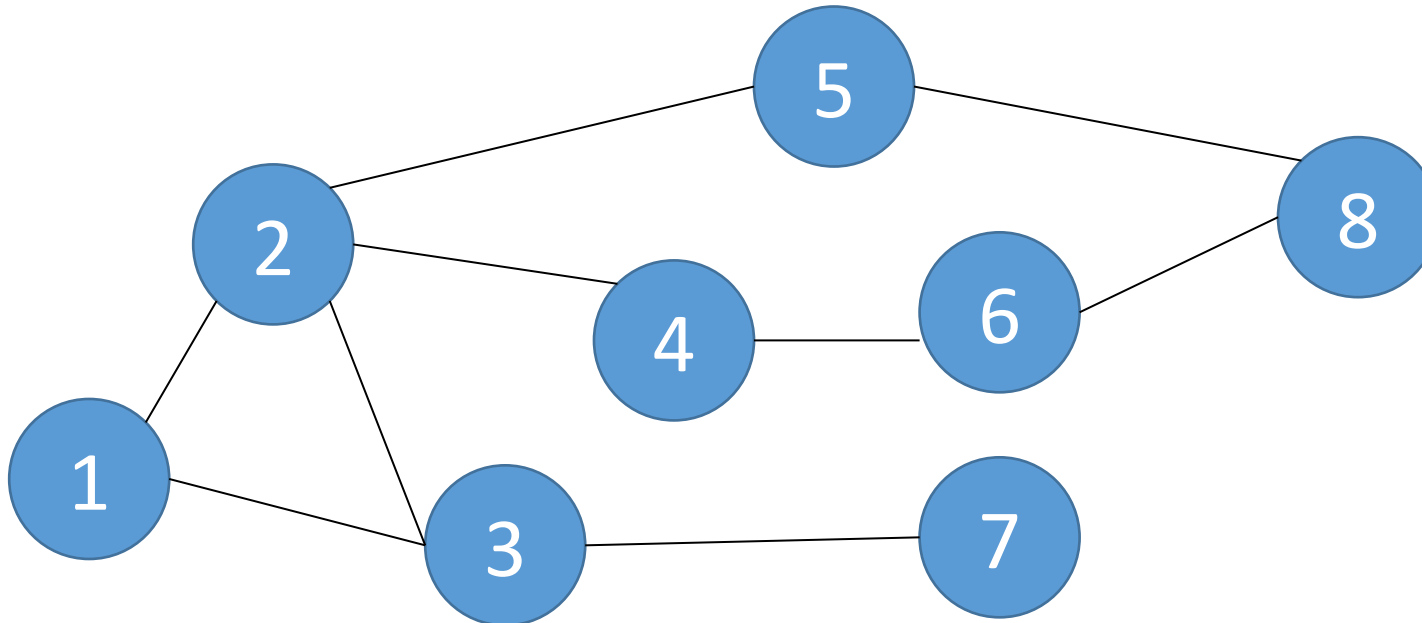
Assumptions: (1) n vertices, m edges

Operations	Adjacency List	Adjacency Matrix
Space	$O(n + m)$	$O(n^2)$
incidentEdges(v)	$O(\deg(v))$	$O(n)$
areAdjacent(v, w)	$O(\min(\deg(v), \deg(w)))$	$O(1)$
insertVertex(v)	$O(1)$	$O(n^2)$
insertEdge(e)	$O(1)$	$O(1)$
removeVertex(v)	$O(\deg(v))$	$O(n^2)$
removeEdge(e)	$O(1)$	$O(1)$



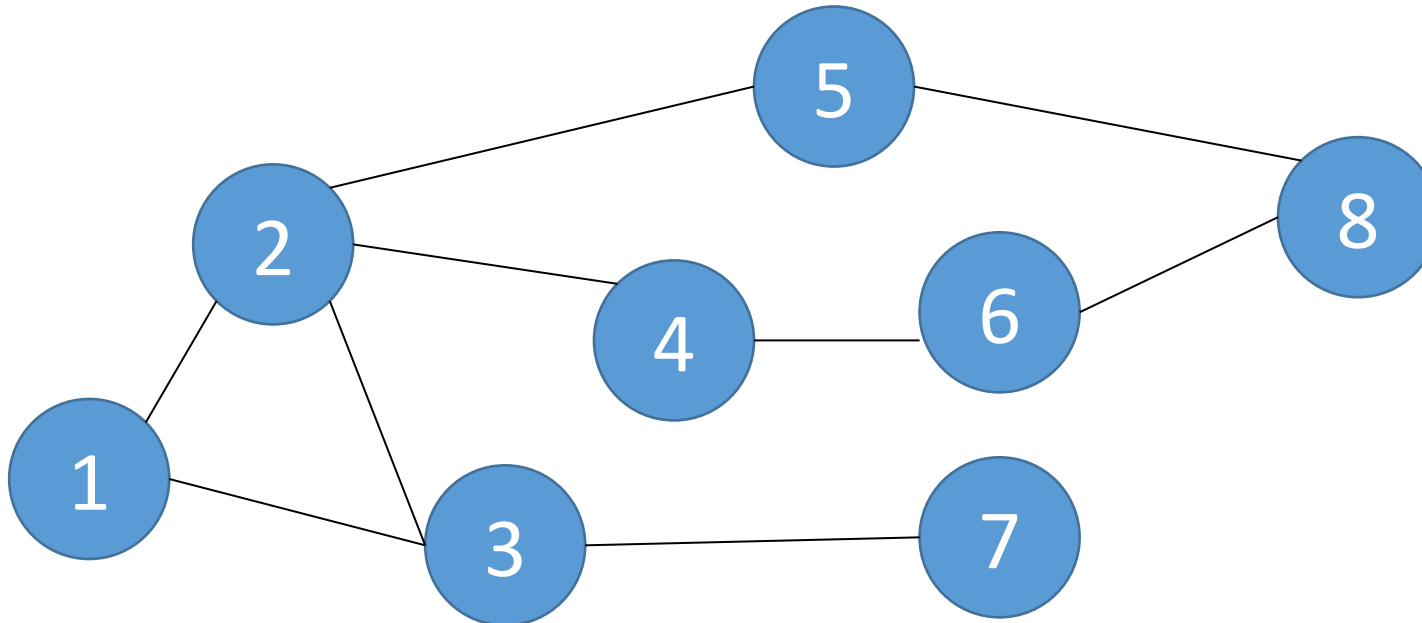
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - Assign a random point to all the adjacent vertices of vertex 2.
 - Assign a color to the adjacent vertices of vertex 2.
 - Assign vertex 2 as the parent of the adjacent vertices of vertex 2.



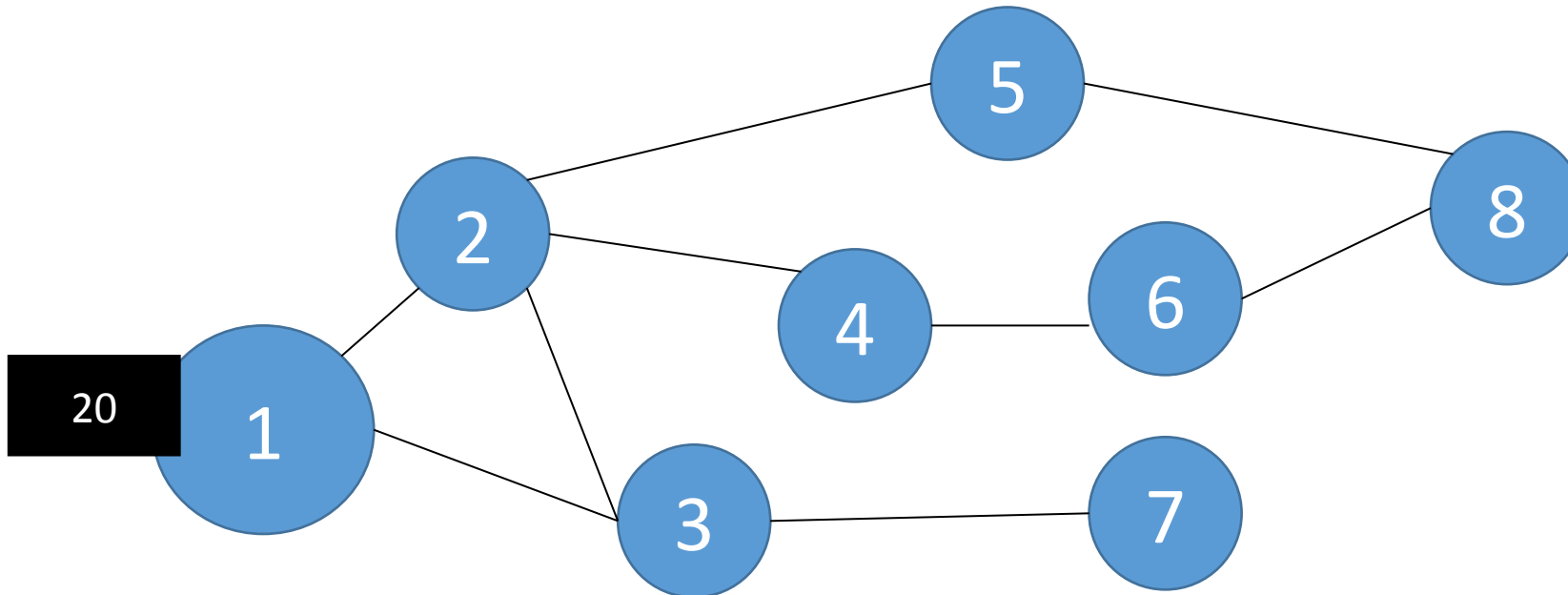
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - **Assign a random point to all the adjacent vertices of vertex 2.**
 - Assign a color to the adjacent vertices of vertex 2.
 - Assign vertex 2 as the parent of the adjacent vertices of vertex 2.



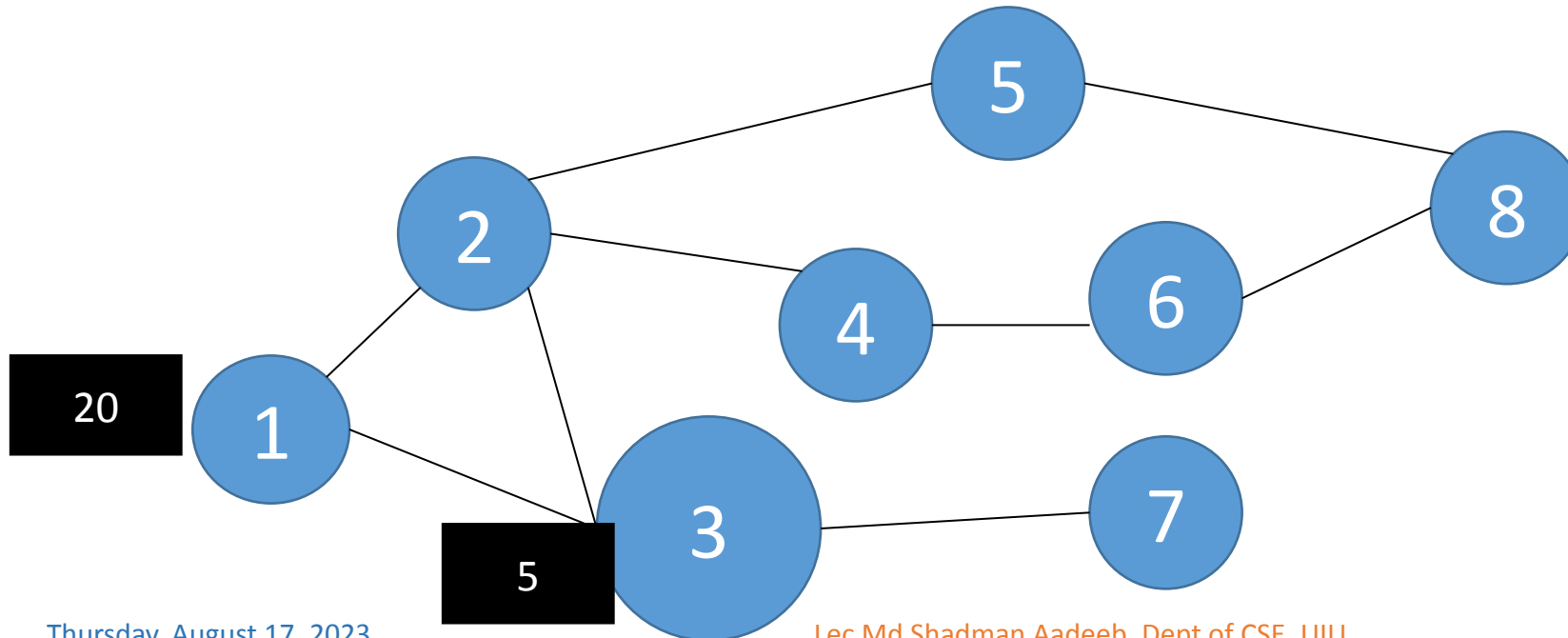
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - **Assign a random point to all the adjacent vertices of vertex 2.**
 - Assign a color to the adjacent vertices of vertex 2.
 - Assign vertex 2 as the parent of the adjacent vertices of vertex 2.



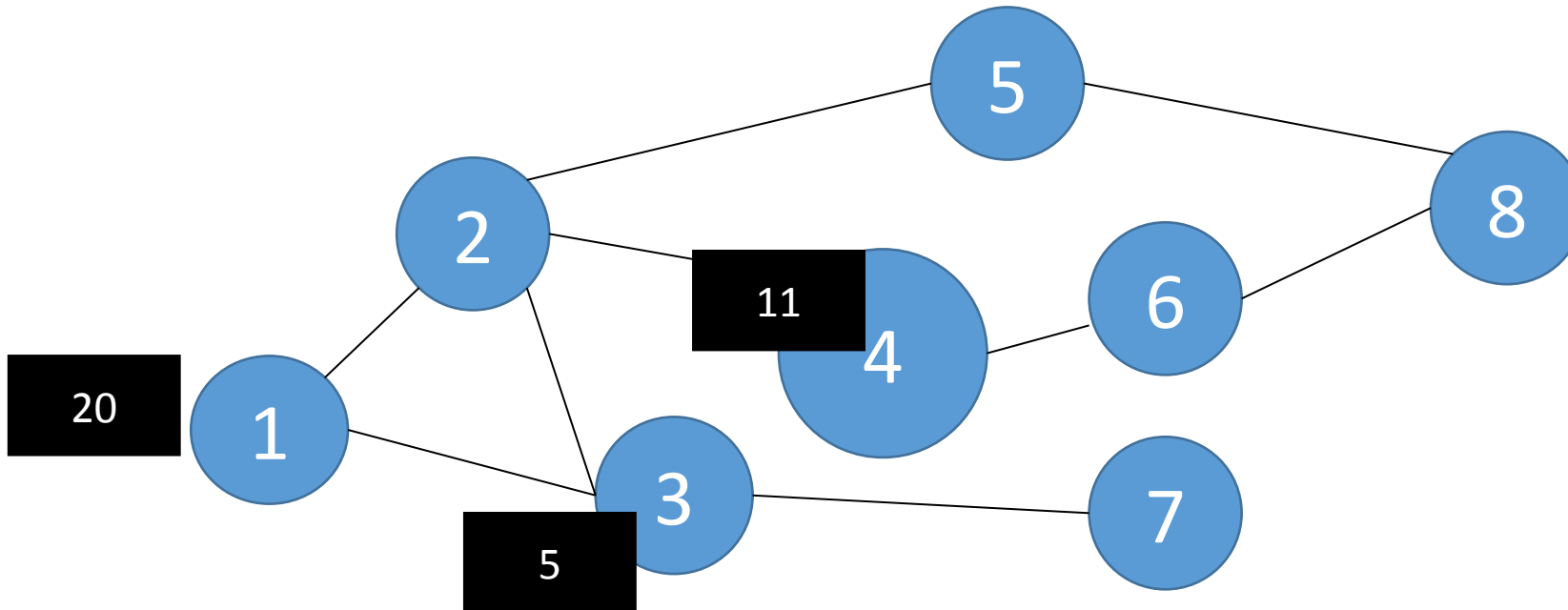
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - **Assign a random point to all the adjacent vertices of vertex 2.**
 - Assign a color to the adjacent vertices of vertex 2.
 - Assign vertex 2 as the parent of the adjacent vertices of vertex 2.



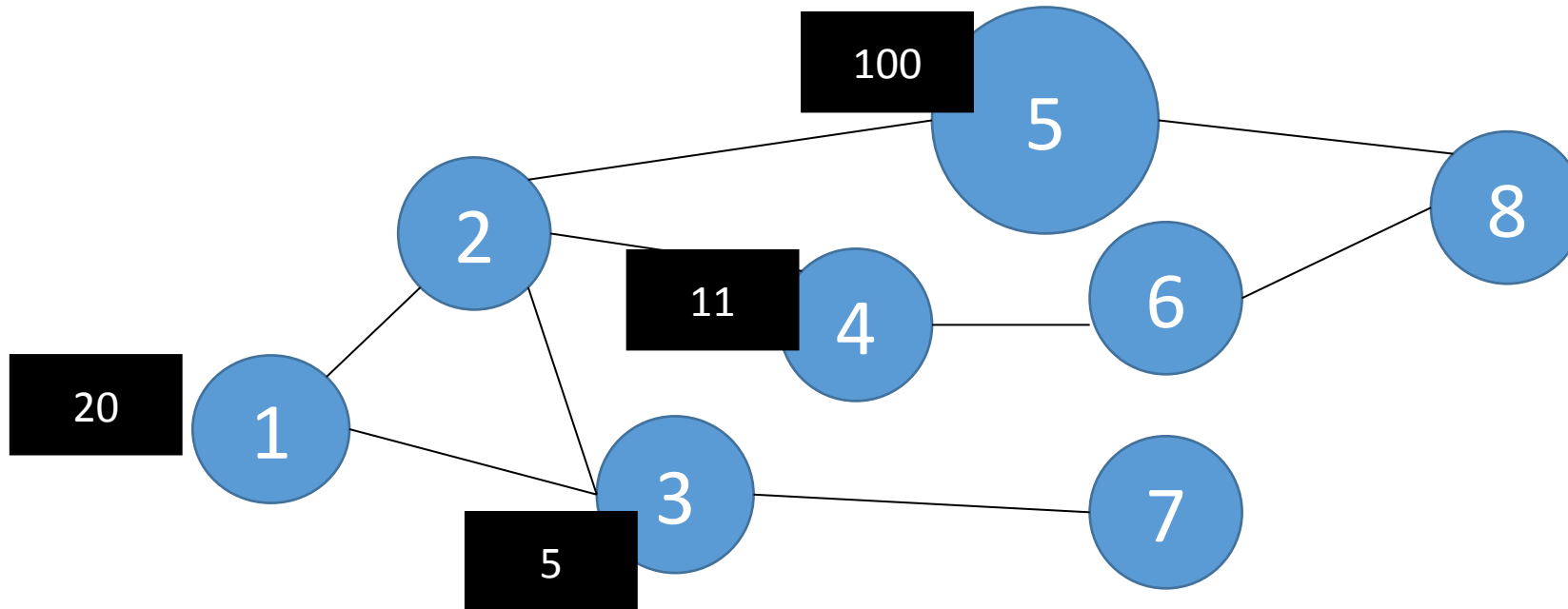
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - **Assign a random point to all the adjacent vertices of vertex 2.**
 - Assign a color to the adjacent vertices of vertex 2.
 - Assign vertex 2 as the parent of the adjacent vertices of vertex 2.



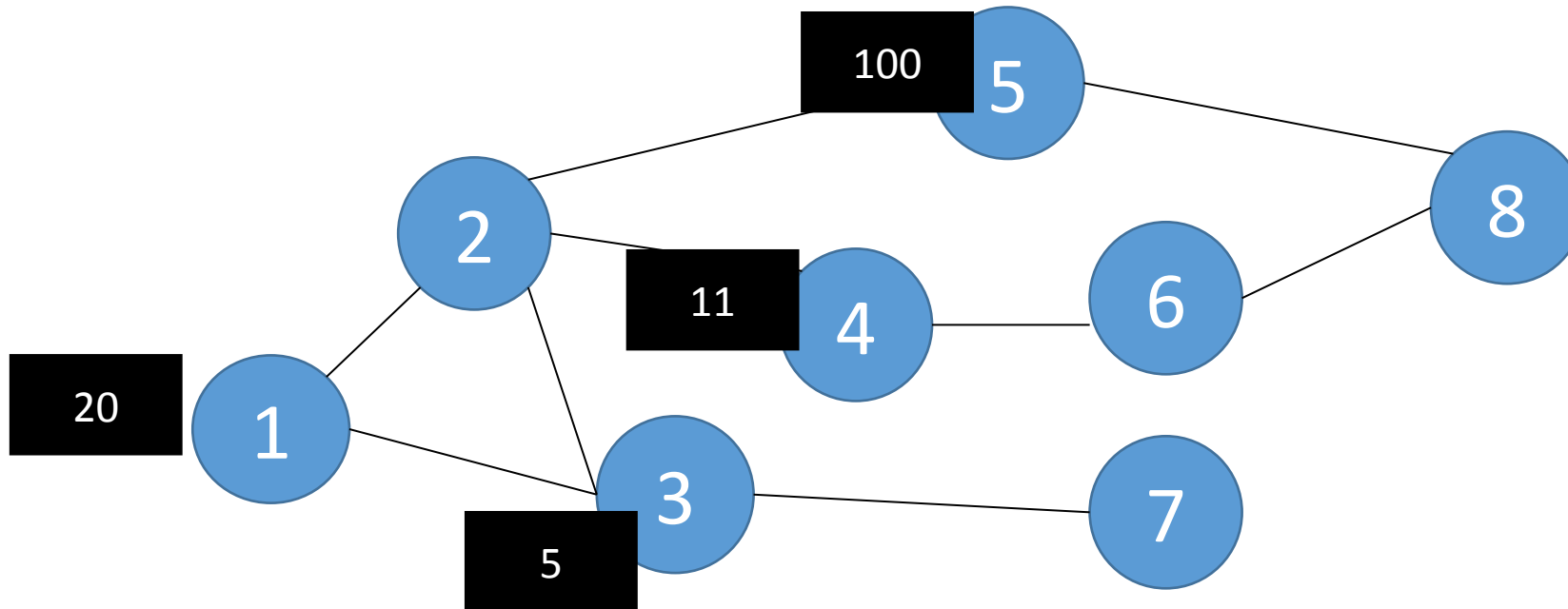
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - **Assign a random point to all the adjacent vertices of vertex 2.**
 - Assign a color to the adjacent vertices of vertex 2.
 - Assign vertex 2 as the parent of the adjacent vertices of vertex 2.



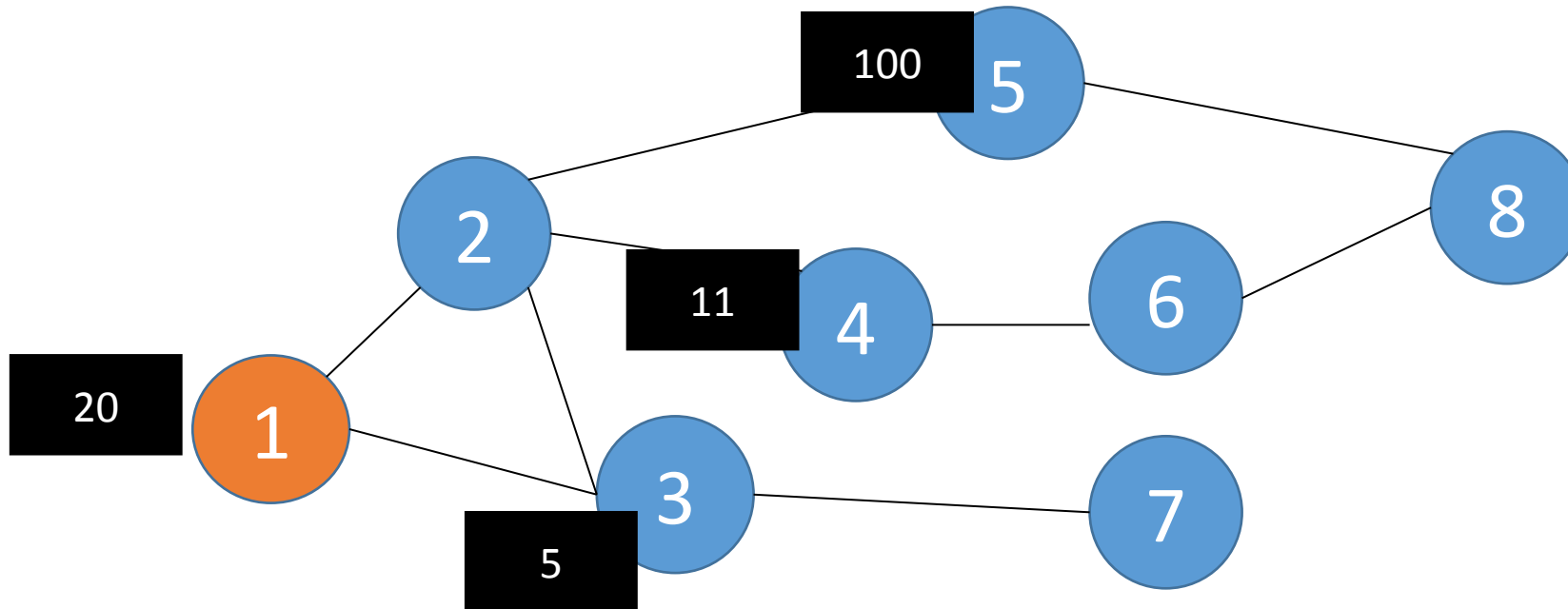
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - Assign a random point to all the adjacent vertices of vertex 2.
 - Assign a color to the adjacent vertices of vertex 2.
 - Assign vertex 2 as the parent of the adjacent vertices of vertex 2.



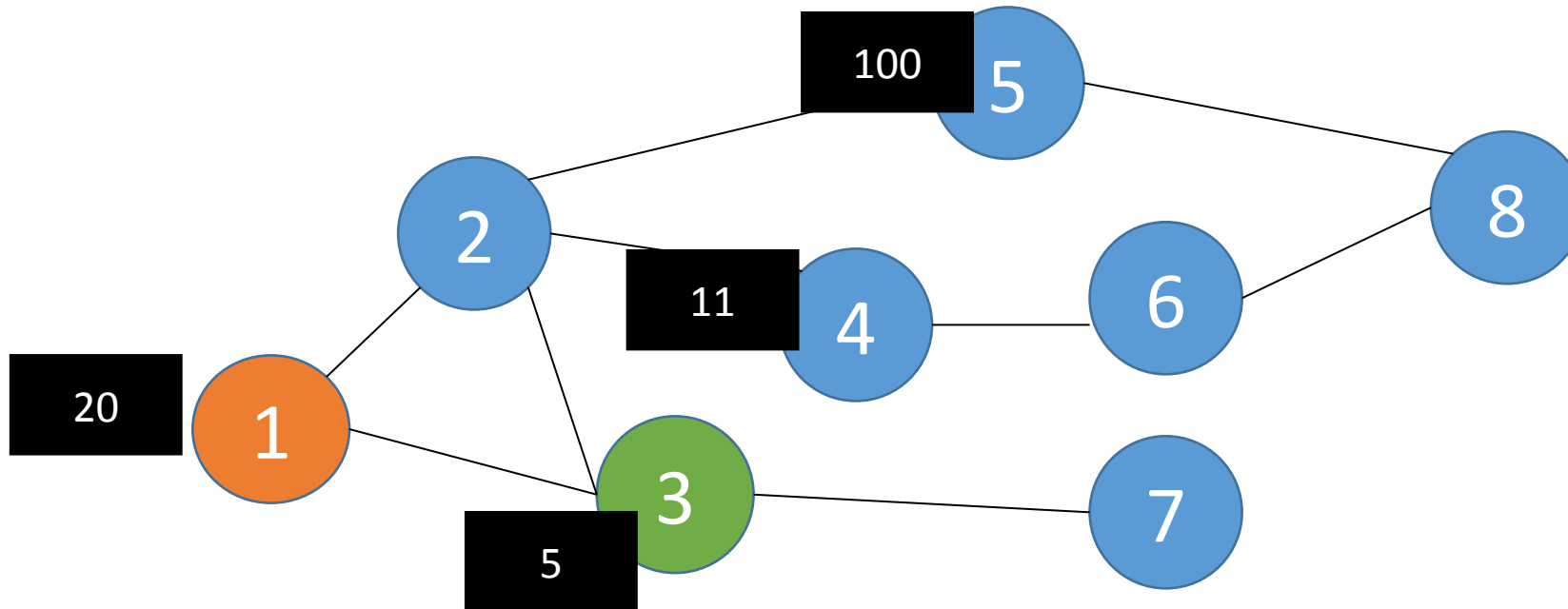
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - Assign a random point to all the adjacent vertices of vertex 2.
 - **Assign a color to the adjacent vertices of vertex 2.**
 - Assign vertex 2 as the parent of the adjacent vertices of vertex 2.



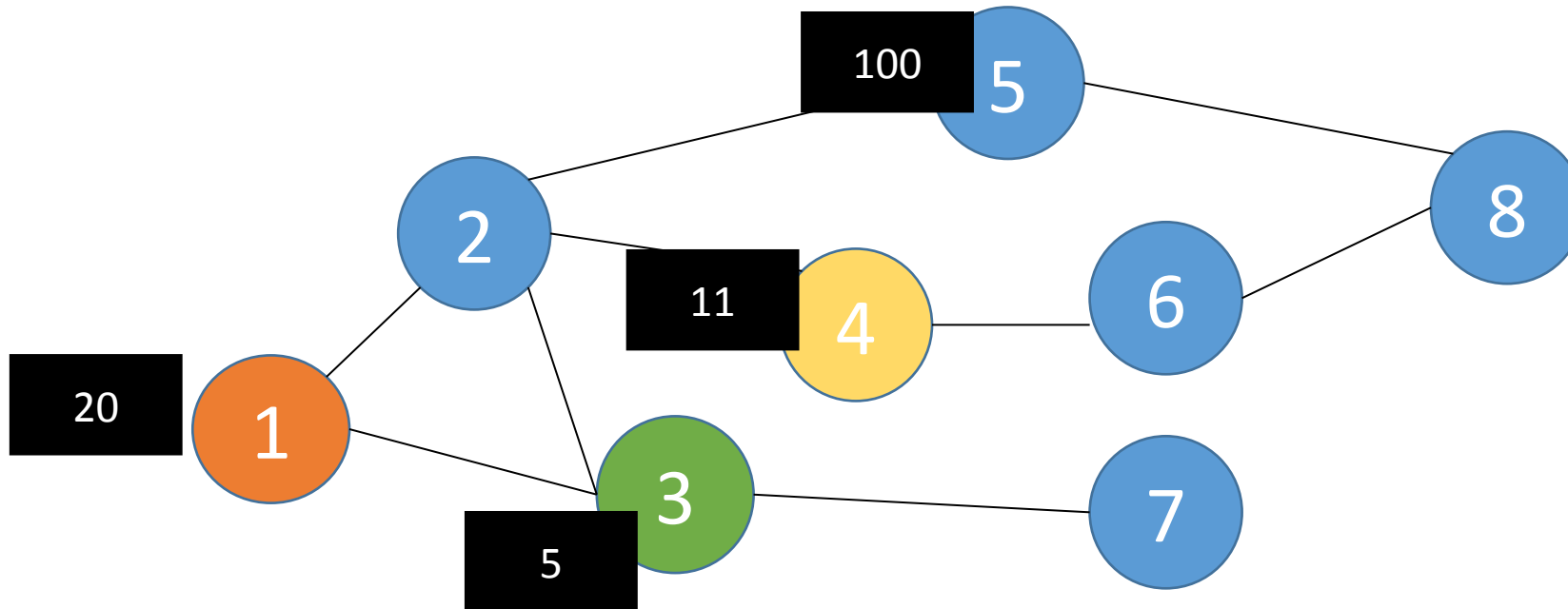
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - Assign a random point to all the adjacent vertices of vertex 2.
 - **Assign a color to the adjacent vertices of vertex 2.**
 - Assign vertex 2 as the parent of the adjacent vertices of vertex 2.



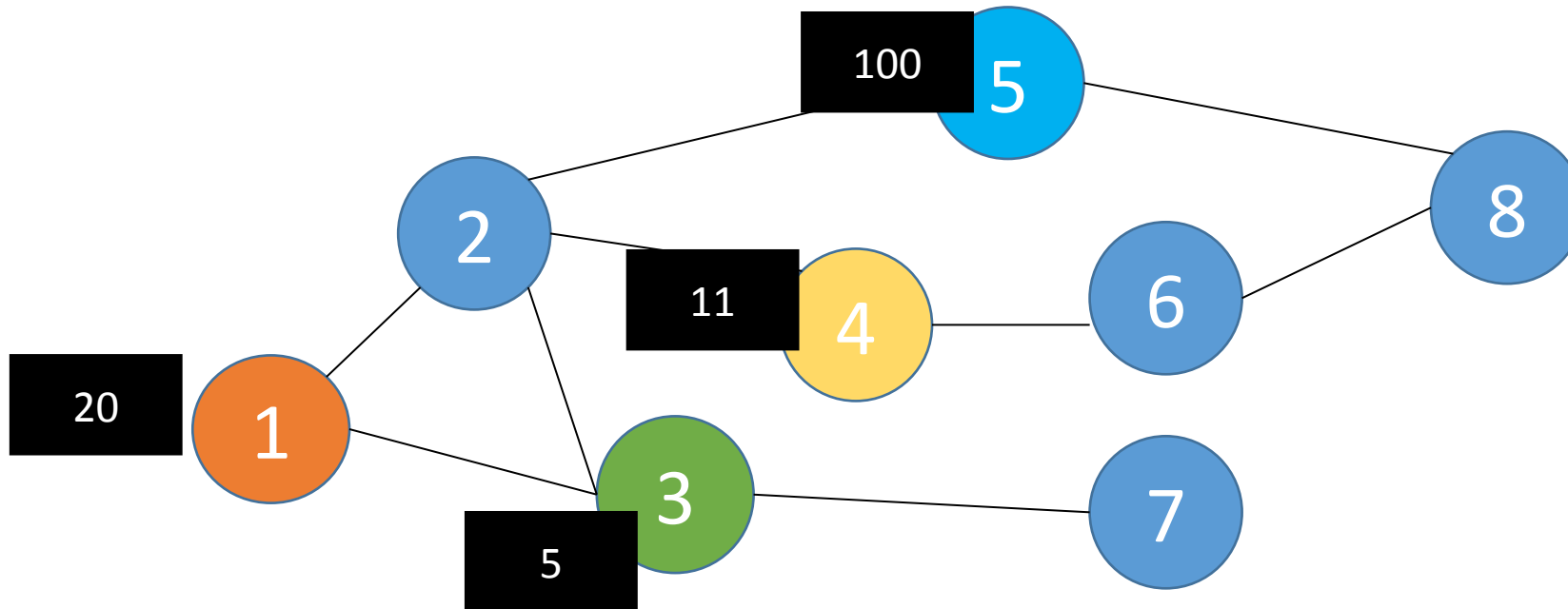
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - Assign a random point to all the adjacent vertices of vertex 2.
 - **Assign a color to the adjacent vertices of vertex 2.**
 - Assign vertex 2 as the parent of the adjacent vertices of vertex 2.



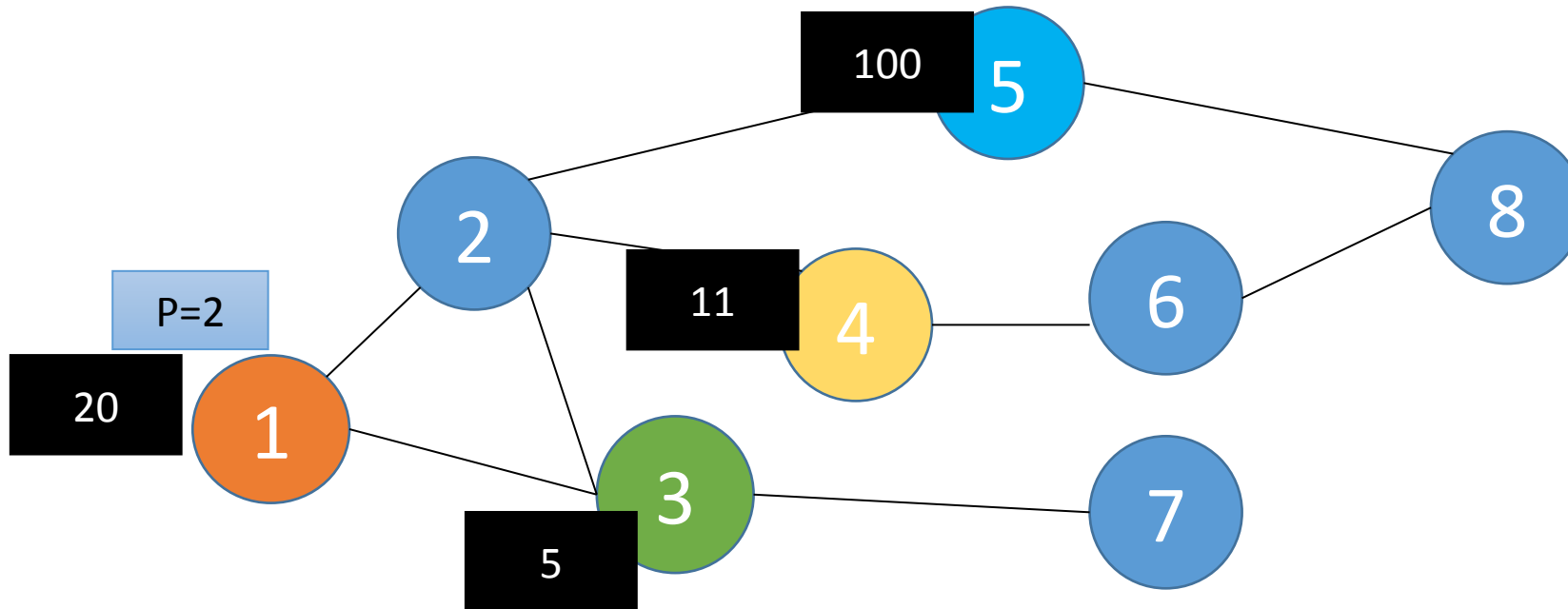
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - Assign a random point to all the adjacent vertices of vertex 2.
 - **Assign a color to the adjacent vertices of vertex 2.**
 - Assign vertex 2 as the parent of the adjacent vertices of vertex 2.



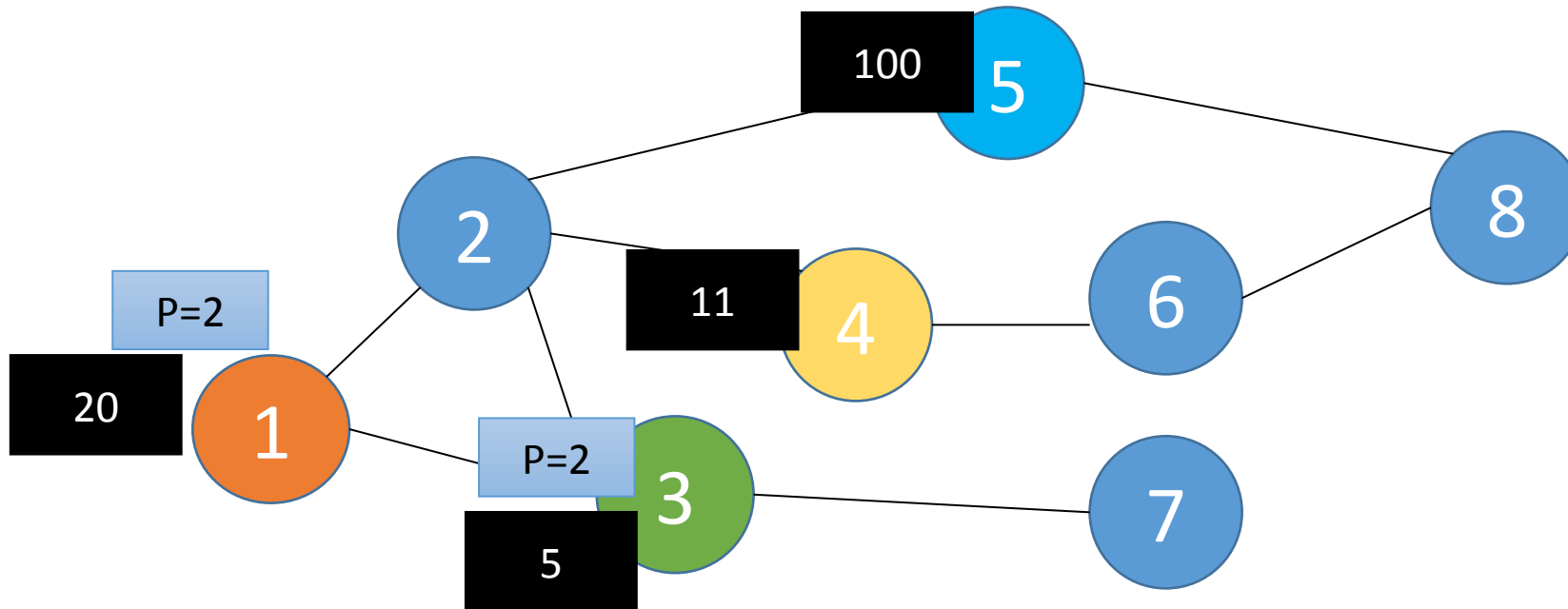
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - Assign a random point to all the adjacent vertices of vertex 2.
 - Assign a color to the adjacent vertices of vertex 2.
 - **Assign vertex 2 as the parent of the adjacent vertices of vertex 2.**



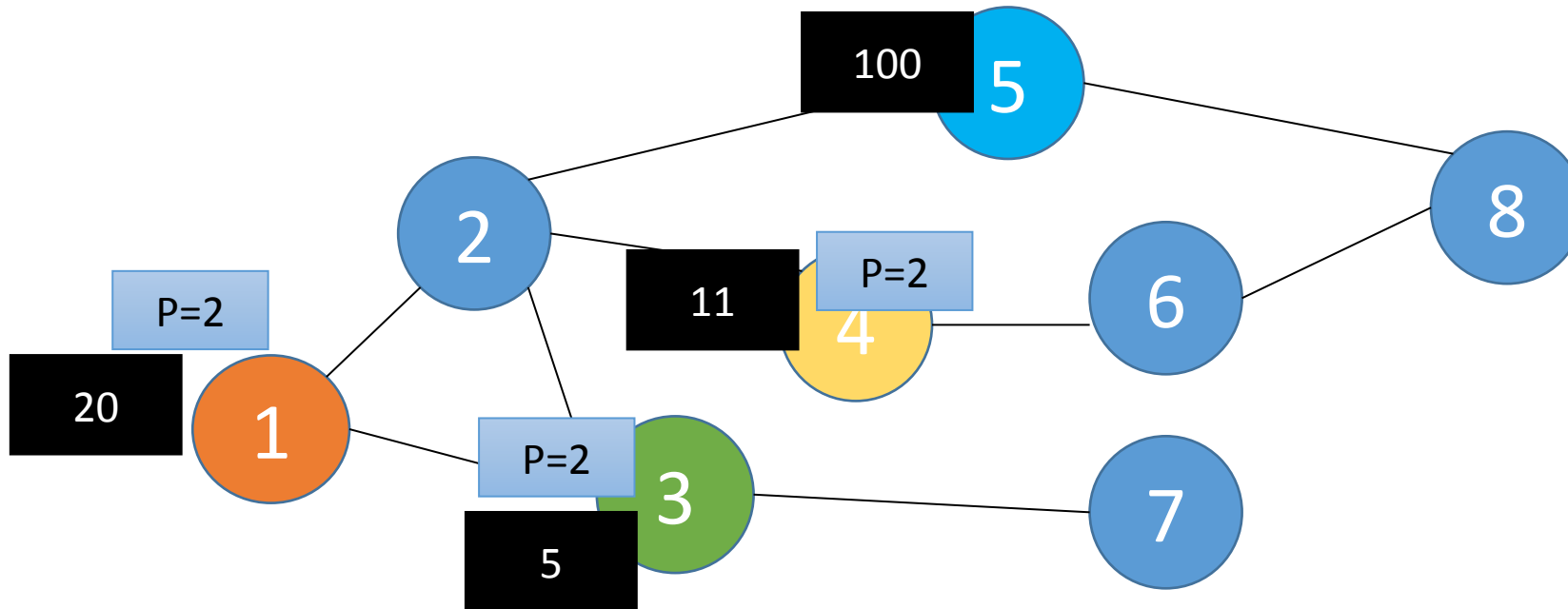
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - Assign a random point to all the adjacent vertices of vertex 2.
 - Assign a color to the adjacent vertices of vertex 2.
 - **Assign vertex 2 as the parent of the adjacent vertices of vertex 2.**



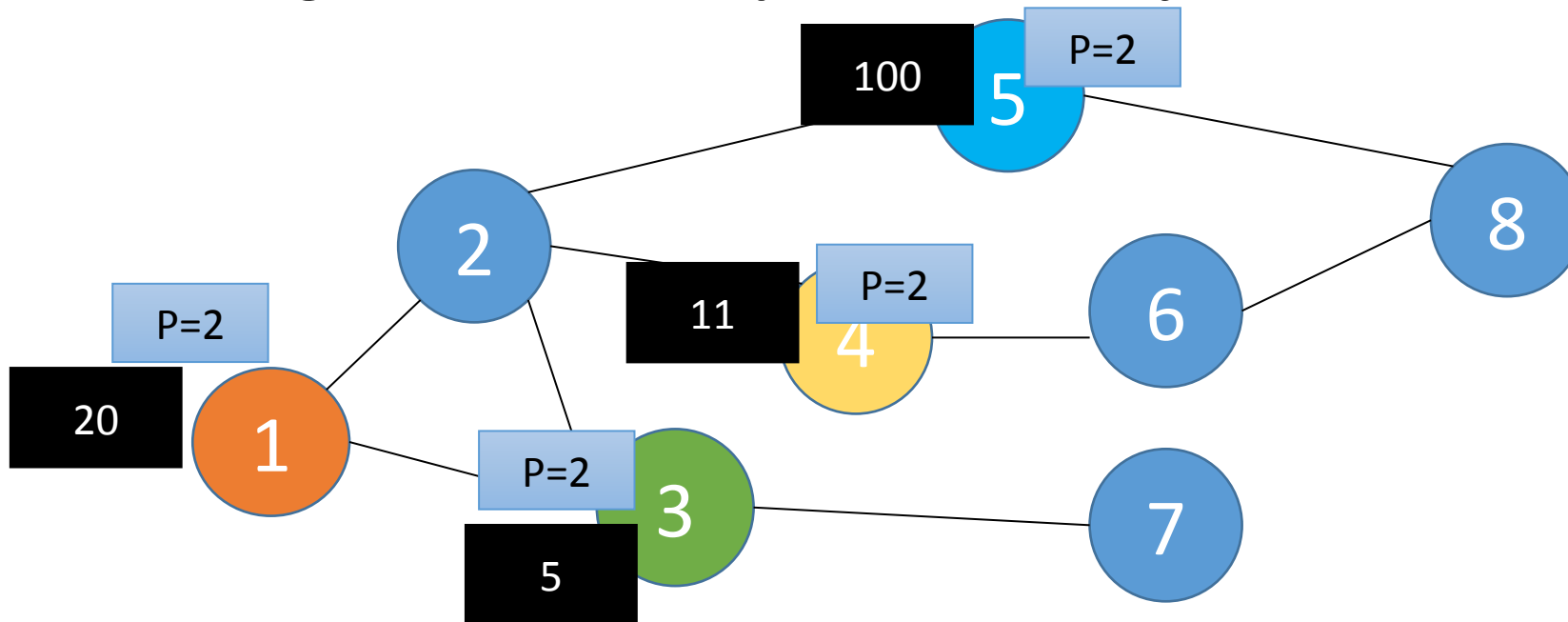
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - Assign a random point to all the adjacent vertices of vertex 2.
 - Assign a color to the adjacent vertices of vertex 2.
 - **Assign vertex 2 as the parent of the adjacent vertices of vertex 2.**



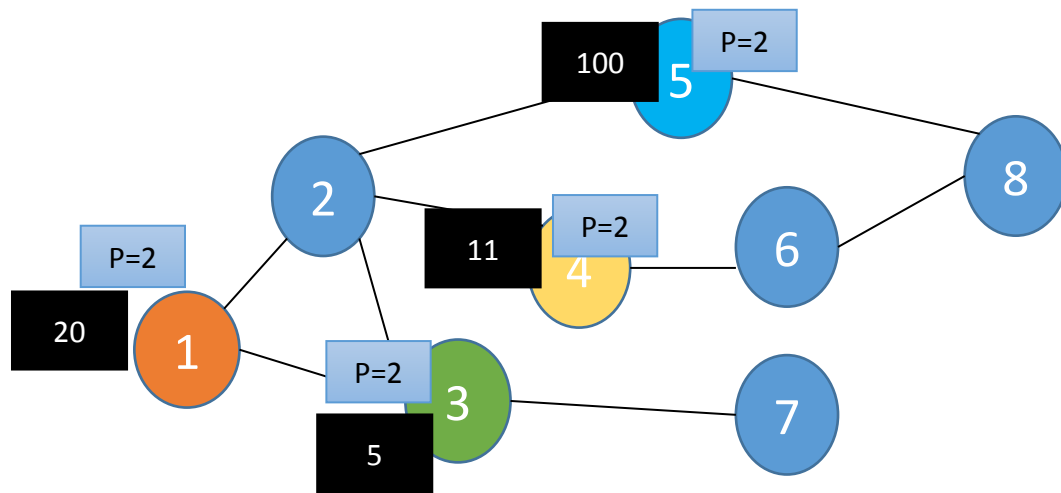
How to think about these representations?

- Consider the following graph, drawn on a piece of paper.
- Do the following:
 - Assign a random point to all the adjacent vertices of vertex 2.
 - Assign a color to the adjacent vertices of vertex 2.
 - **Assign vertex 2 as the parent of the adjacent vertices of vertex 2.**



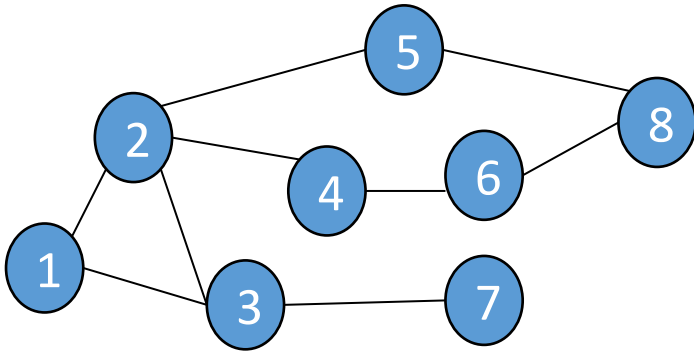
How to think about these representations?

- We just did the task on paper
- Let us try to denote it using an extremely simple pseudocode



```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



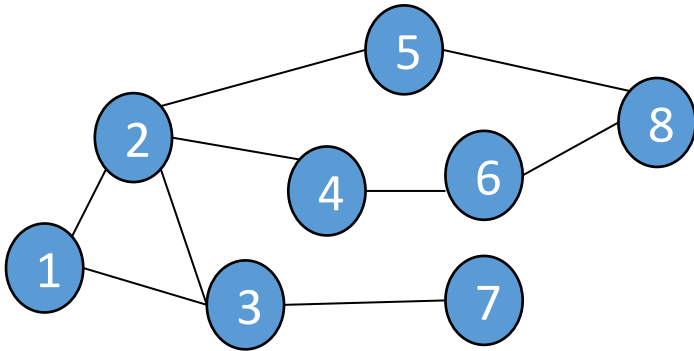
Parent



Adjacency Matrix

	1	2	3	4	5	6	7	8
1		1	1	1	1			
2	1		1					1
3		1					1	
4								1
5		1						1
6				1				
7			1					
8					1	1		

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent

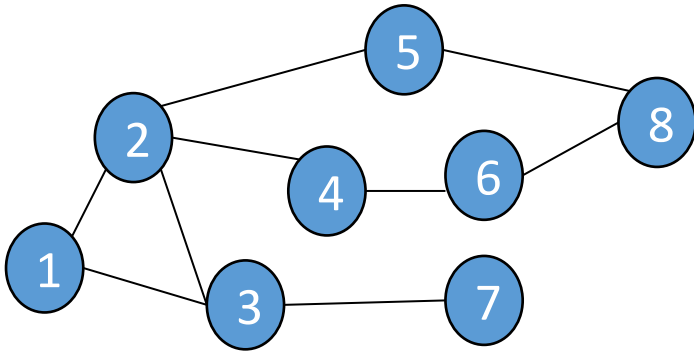


Adjacency Matrix

	1	1					
1		1	1	1			
1	1					1	
	1				1		
	1		1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```


Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent

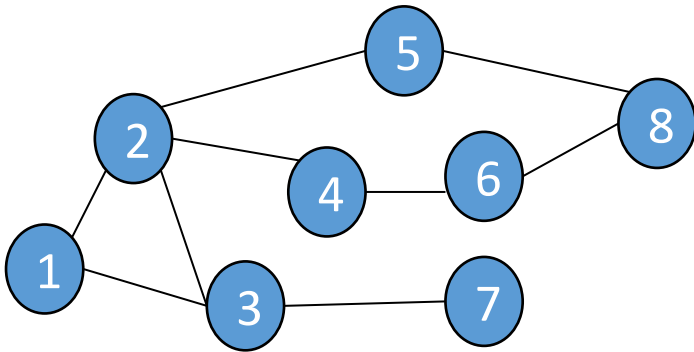


Adjacency Matrix

	1	2	3	4	5	6	7	8
1		1						
2	1		1	1	1			
3		1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



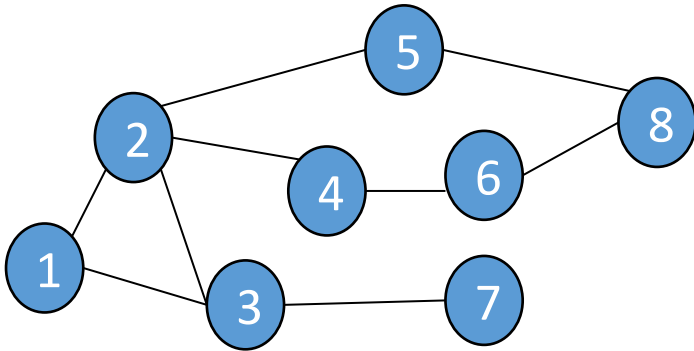
Adjacency Matrix

	1	1					
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int i=1;i<=8;i++){  
  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



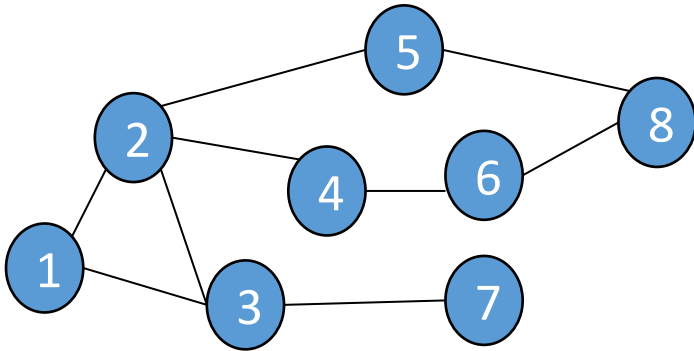
Adjacency Matrix

	1	1					
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    printf("%d",j);  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



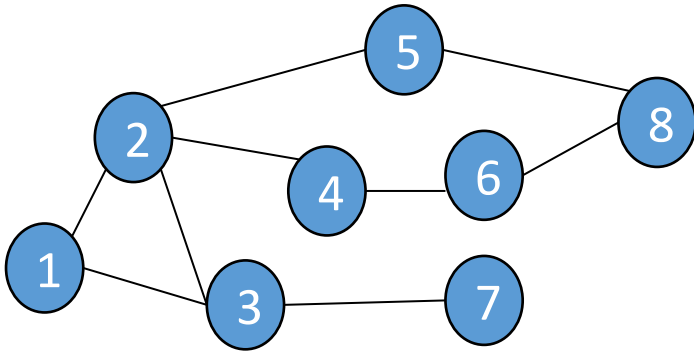
Adjacency Matrix

	1	1					
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    printf("%d",j);  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



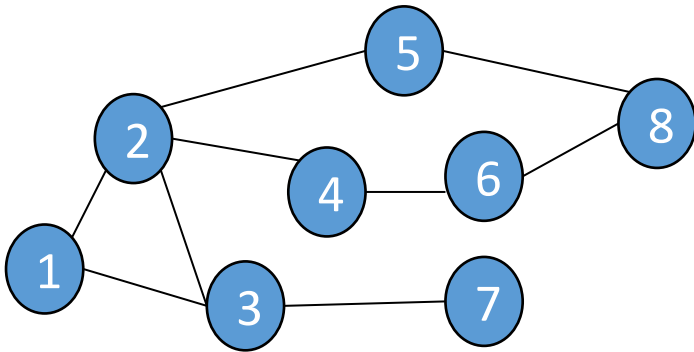
Adjacency Matrix

		1					
1	1	1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    printf("%d",j);  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

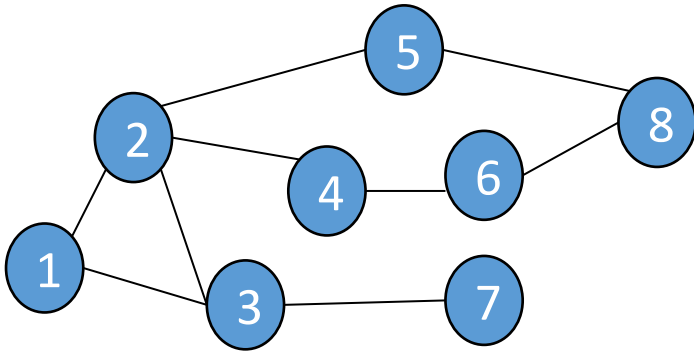
	1	1					
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    printf("%d",j);  
}
```

1 2 3

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

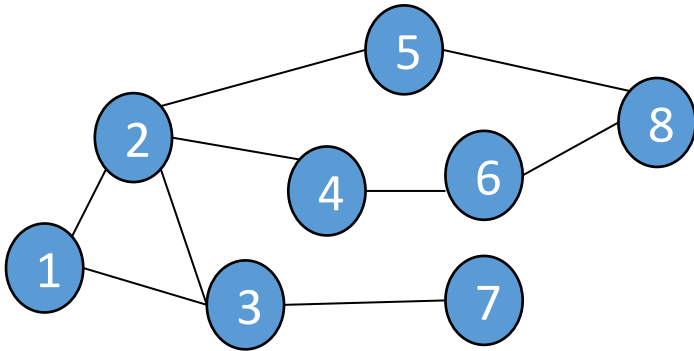
	1	1					
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    printf("%d",j);  
}
```

1 2 3 4

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

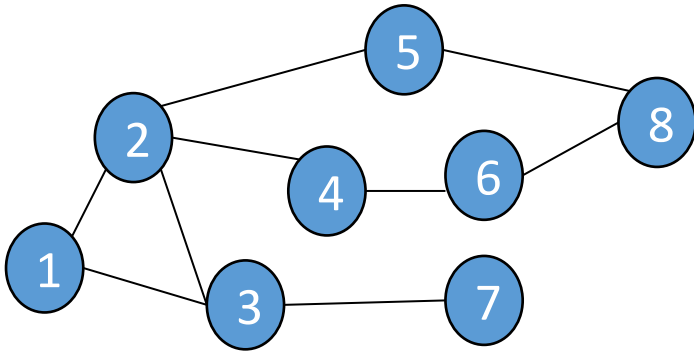
	1	2	3	4	5	6	7	8
1		1	1	1	1			
2	1		1				1	
3		1						
4		1						1
5				1				1
6			1					
7								
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    printf("%d",j);  
}
```

1 2 3 4 5

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

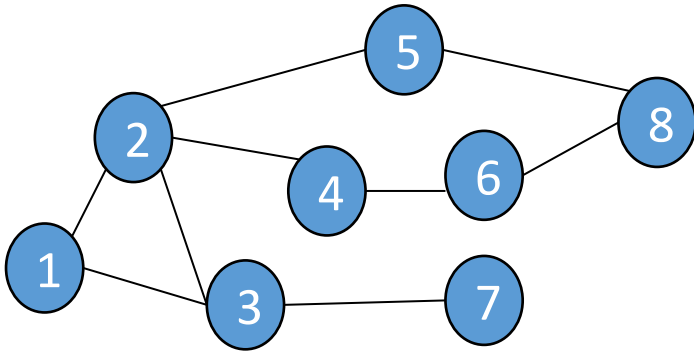
	1	1					
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    printf("%d",j);  
}
```

1 2 3 4 5 6

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

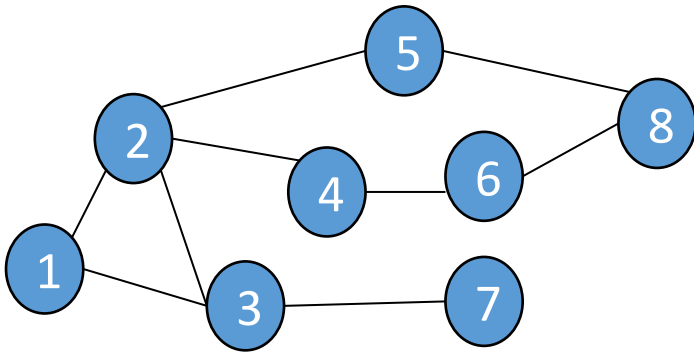
	1	1					
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    printf("%d",j);  
}
```

1 2 3 4 5 6 7

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

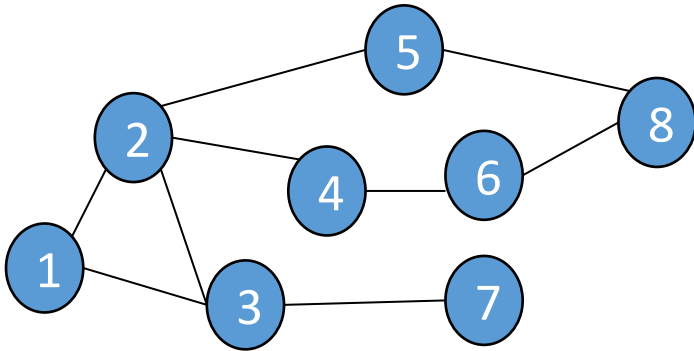
	1	2	3	4	5	6	7	8
1		1	1	1	1			
2	1						1	
3		1				1		
4		1						1
5				1				1
6			1					
7								
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    printf("%d",j);  
}
```

1 2 3 4 5 6 7 8

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



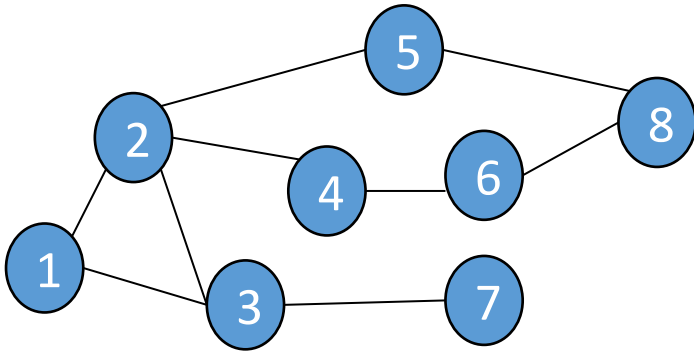
Adjacency Matrix

	1	1					
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        printf("%d",j);  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



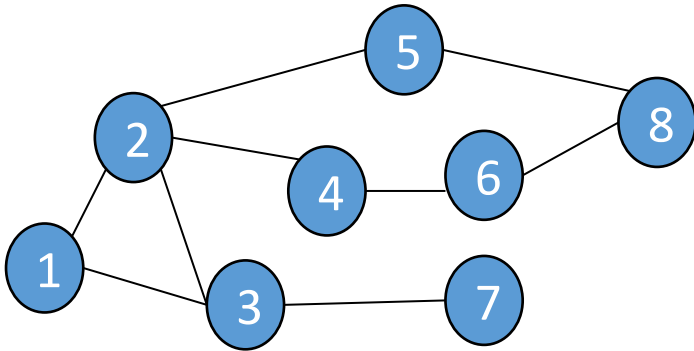
Adjacency Matrix

	1	1					
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if (Adj[2][j]==1) {  
        printf("%d",j);  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



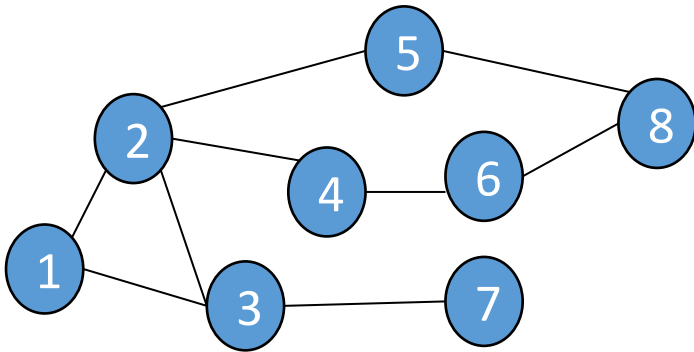
Adjacency Matrix

		1					
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        printf("%d",j);  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

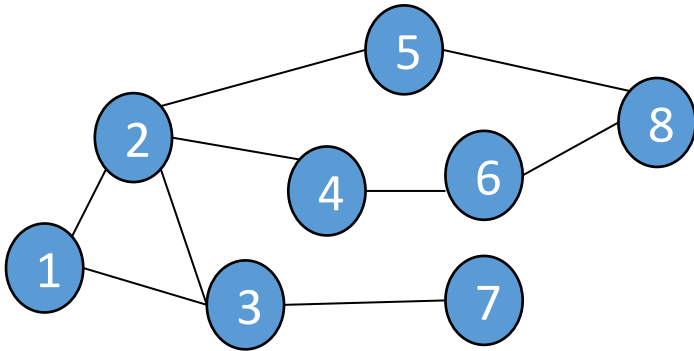
	1	1	↓				
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if (Adj[2][j]==1) {  
    printf("%d",j);  
    }  
}
```

1 3 4

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

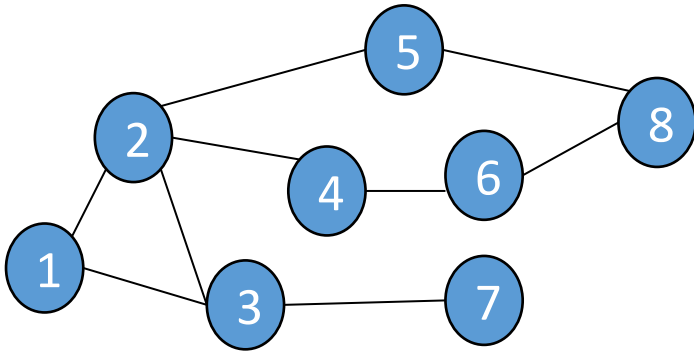
	1	1					
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if (Adj[2][j]==1) {  
    printf("%d",j);  
    }  
}
```

1 3 4 5

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



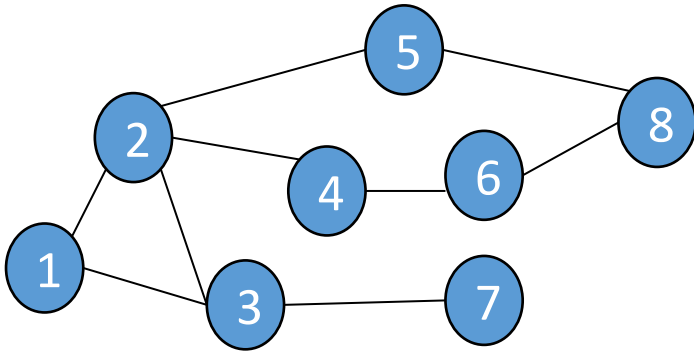
Adjacency Matrix

	1	2	3	4	5	6	7	8
1		1						
2	1		1	1	1			
3		1						
4								
5								
6								
7								
8								

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if (Adj[2][j]==1){  
  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



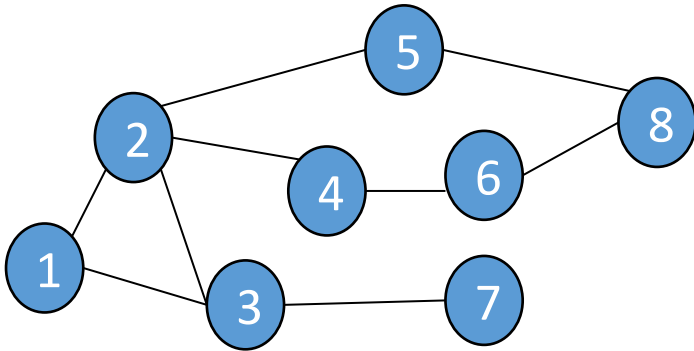
Adjacency Matrix

	1	2	3	4	5	6	7	8
1		1	1	1	1			
2	1		1					1
3		1					1	
4		1						1
5				1				1
6			1					
7								
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



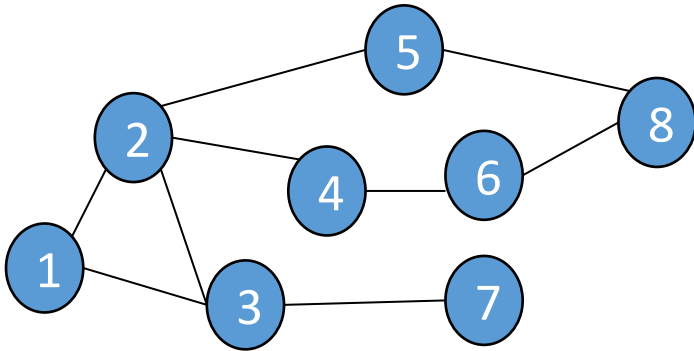
Adjacency Matrix

	1	2	3	4	5	6	7	8
1		1	1	1				
2	1		1					1
3		1					1	
4		1						1
5				1				1
6			1					
7								
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



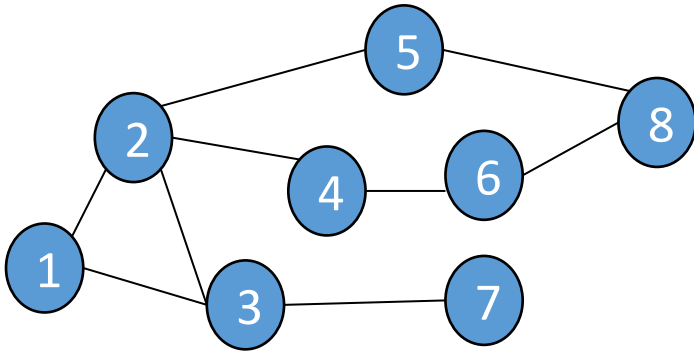
Adjacency Matrix

	1	1					
1		1	1	1			
1	1					1	
	1				1		
	1						1
			1				1
		1					
				1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



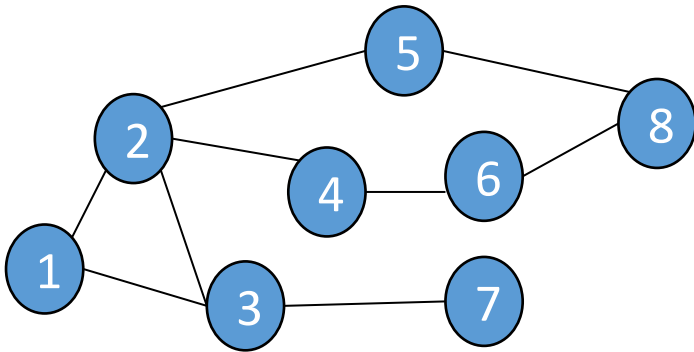
Adjacency Matrix

	1	2	3	4	5	6	7	8
1		1	1	1				
2	1						1	
3		1						
4		1						1
5				1				1
6			1					
7								
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



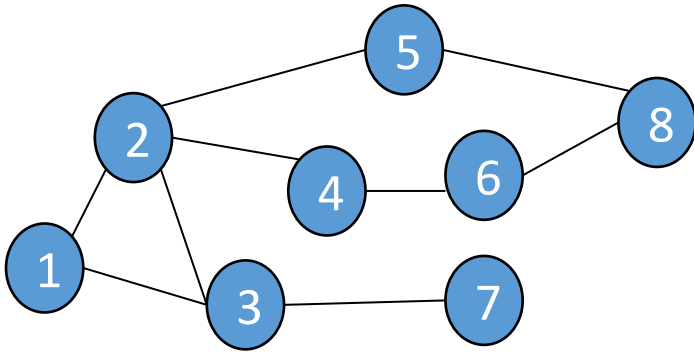
Adjacency Matrix

	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



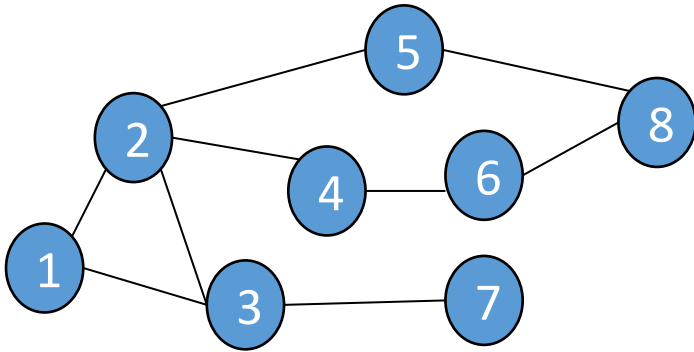
Adjacency Matrix

	j	1	2	3	4	5	6	7	8
1			1	1					
2		1		1	1	1			
3		1	1					1	
4			1				1		
5			1						1
6					1				1
7				1					
8						1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if (Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



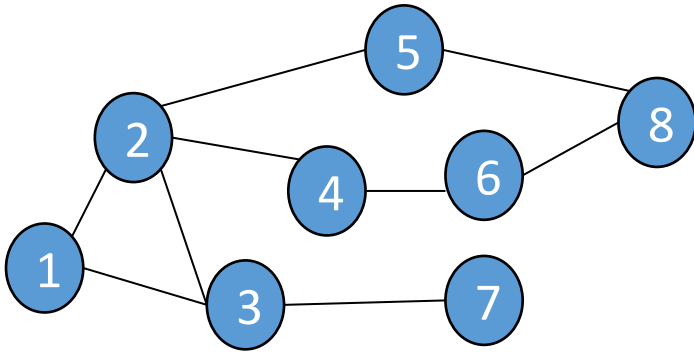
Adjacency Matrix

	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if (Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```


Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



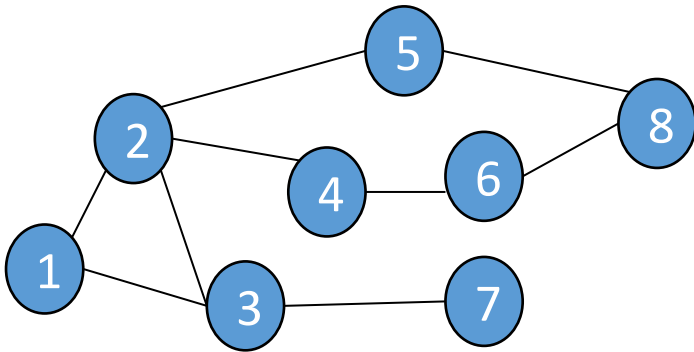
Adjacency matrix

	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if (Adj[2][j]==1) {  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



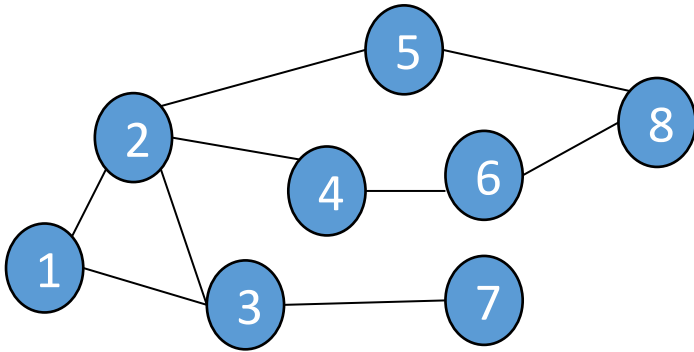
Adjacency Mat $j \times$

	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if (Adj[2][j]==1) {  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



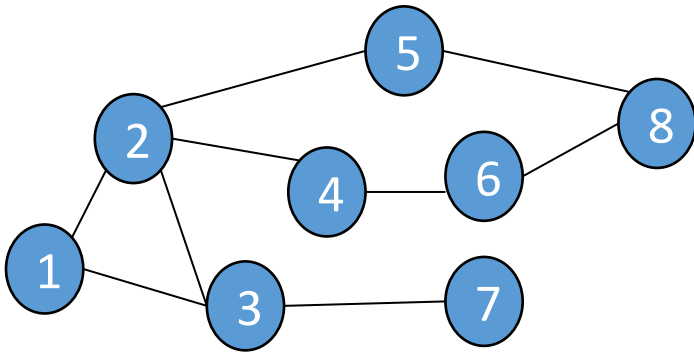
Adjacency Matrix

	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if (Adj[2][j]==1) {  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

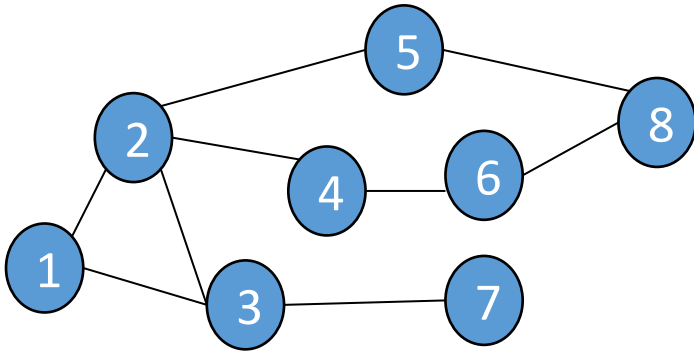
↓ j

	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if (Adj[2][j]==1) {  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

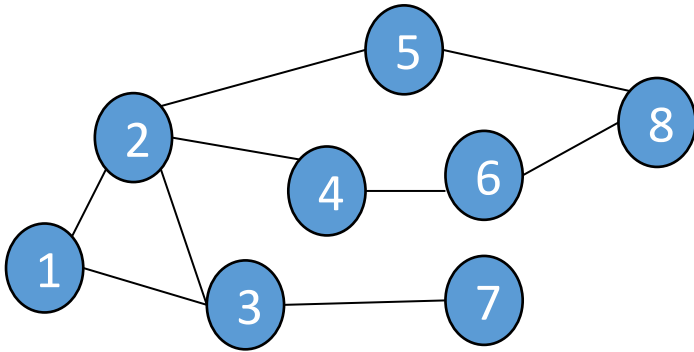
↓ j

	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if (Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



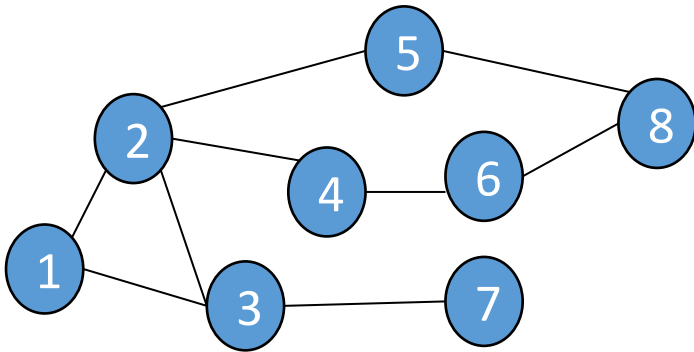
Adjacency Matrix

	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if (Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

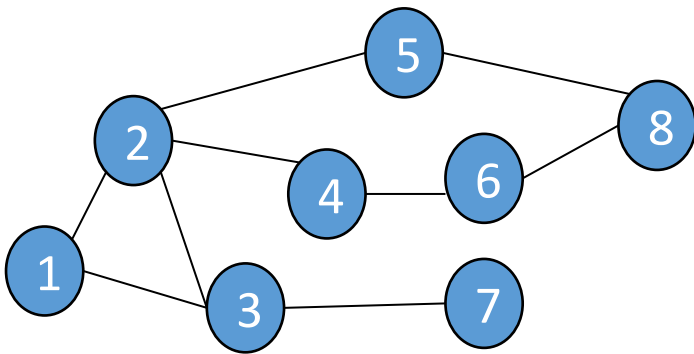
	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

Some work is being done only when this condition is being met

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

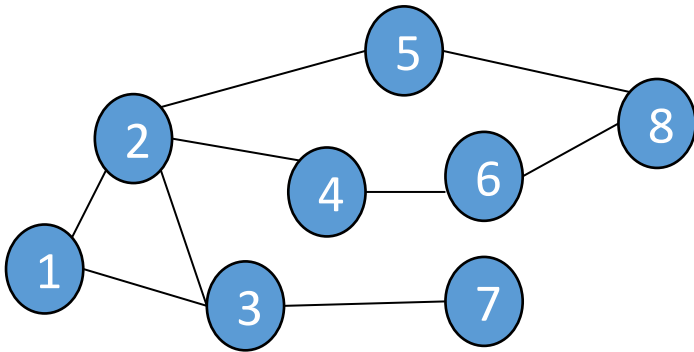
	1	2	3	4	5	6	7	8
1		1						
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

Some work is being done only when this condition is being met

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```


Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

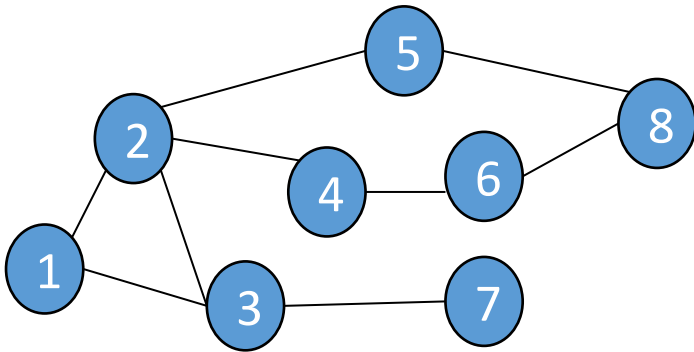
	1	2	3	4	5	6	7	8
1		1	1	1				
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

Some work is being done only when this condition is being met

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

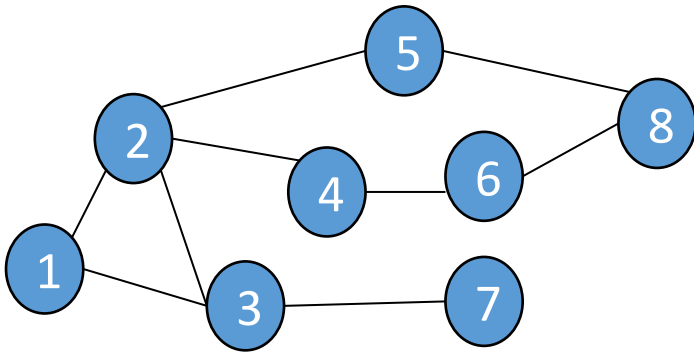
	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

Some work is being done only when this condition is being met

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

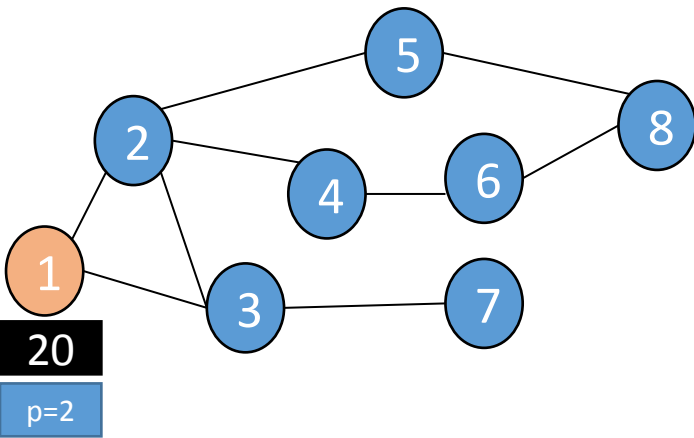
	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

j=1

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score

20							
----	--	--	--	--	--	--	--

Color

5							
---	--	--	--	--	--	--	--

Parent

2							
---	--	--	--	--	--	--	--

Adjacency Matrix

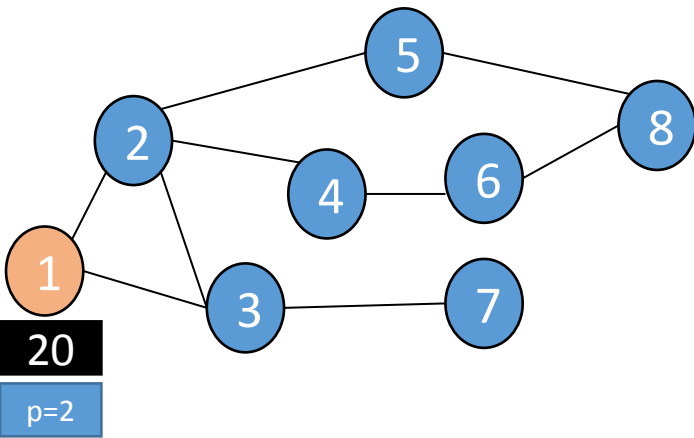
	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

j=1

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score



Color



Parent



Adjacency Matrix

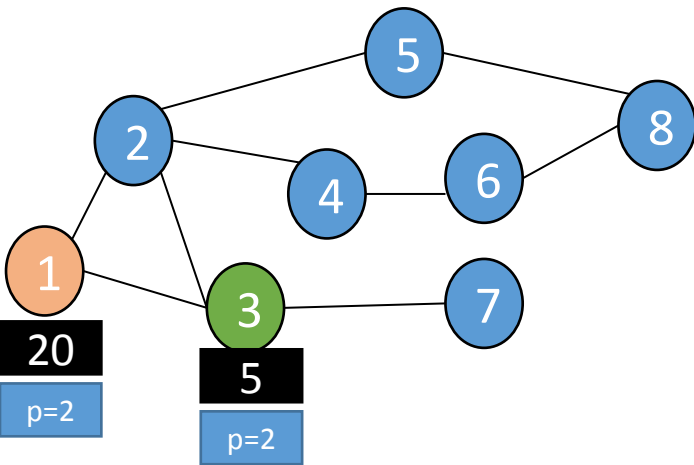
	1	2	3	4	5	6	7	8
1		1						
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

j=3

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score

20		5					
----	--	---	--	--	--	--	--

Color

5		10					
---	--	----	--	--	--	--	--

Parent

2		2					
---	--	---	--	--	--	--	--

Adjacency Matrix

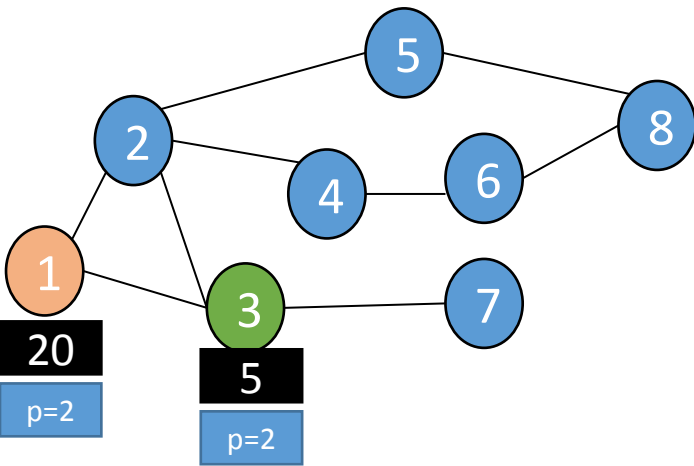
	1	2	3	4	5	6	7	8
1		1						
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

j=3

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score

20		5					
----	--	---	--	--	--	--	--

Color

5		10					
---	--	----	--	--	--	--	--

Parent

2		2					
---	--	---	--	--	--	--	--

Adjacency Matrix

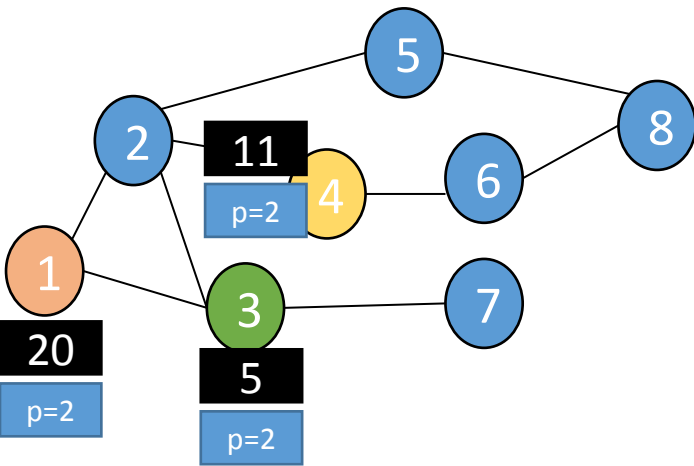
	1	2	3	4	5	6	7	8
1		1	1	1				
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

j=4

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score

20		5	11				
----	--	---	----	--	--	--	--

Color

5		10	7				
---	--	----	---	--	--	--	--

Parent

2		2	2				
---	--	---	---	--	--	--	--

Adjacency Matrix

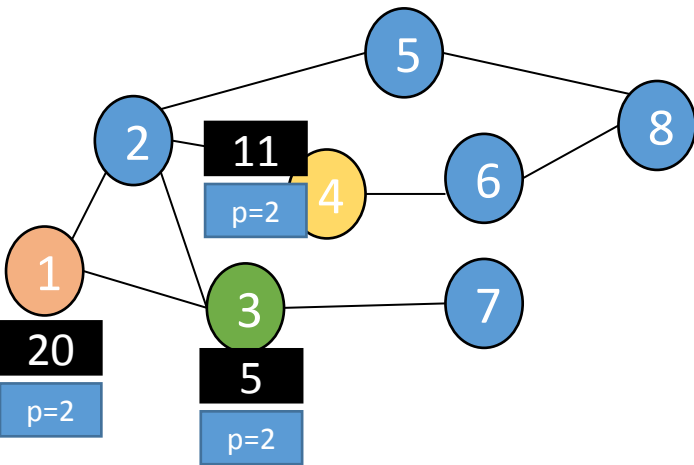
	1	2	3	4	5	6	7	8
1		1	1	1				
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

j=4

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```


Now let us do the same, but using code



Attribute Arrays

Score

20		5	11				
----	--	---	----	--	--	--	--

Color

5		10	7				
---	--	----	---	--	--	--	--

Parent

2		2	2				
---	--	---	---	--	--	--	--

Adjacency Matrix

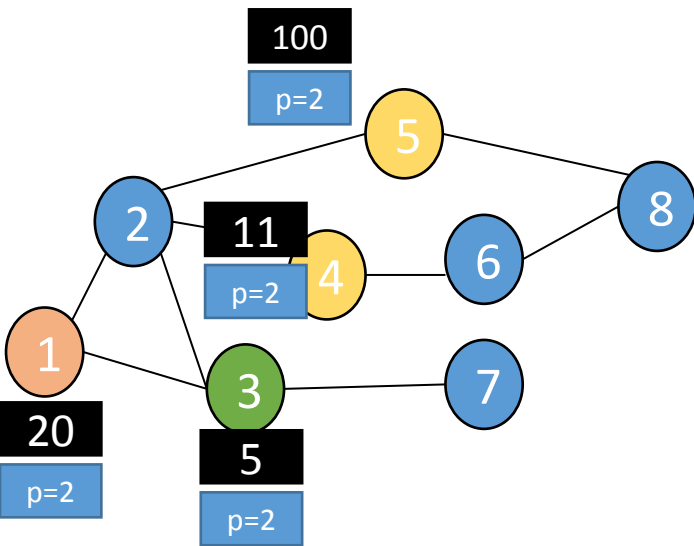
	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

j=5

```
for each vertex v adjacent to 2:  
    v.score=randomNumber()  
    v.color=randomColor()  
    v.parent=2
```

```
for (int j=1;j<=8;j++){  
    if(Adj[2][j]==1){  
        score[j]=random();  
        color[j]=random();  
        parent[j]=2  
    }  
}
```

Now let us do the same, but using code



Attribute Arrays

Score

20		5	11	100			
----	--	---	----	-----	--	--	--

Color

5		10	7	7			
---	--	----	---	---	--	--	--

Parent

2		2	2	2			
---	--	---	---	---	--	--	--

Adjacency Matrix

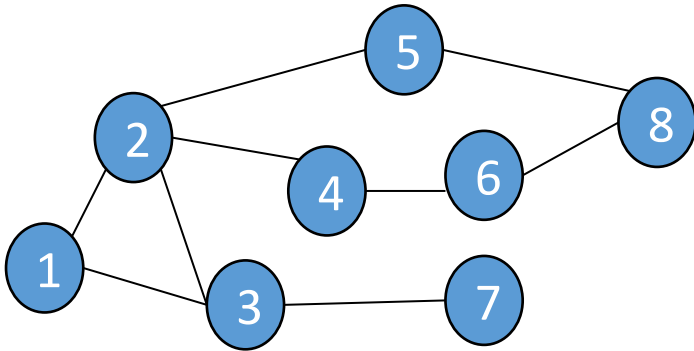
	1	2	3	4	5	6	7	8
1		1	1					
2	1		1	1	1			
3	1	1					1	
4		1				1		
5		1						1
6				1				1
7			1					
8					1	1		

j=5

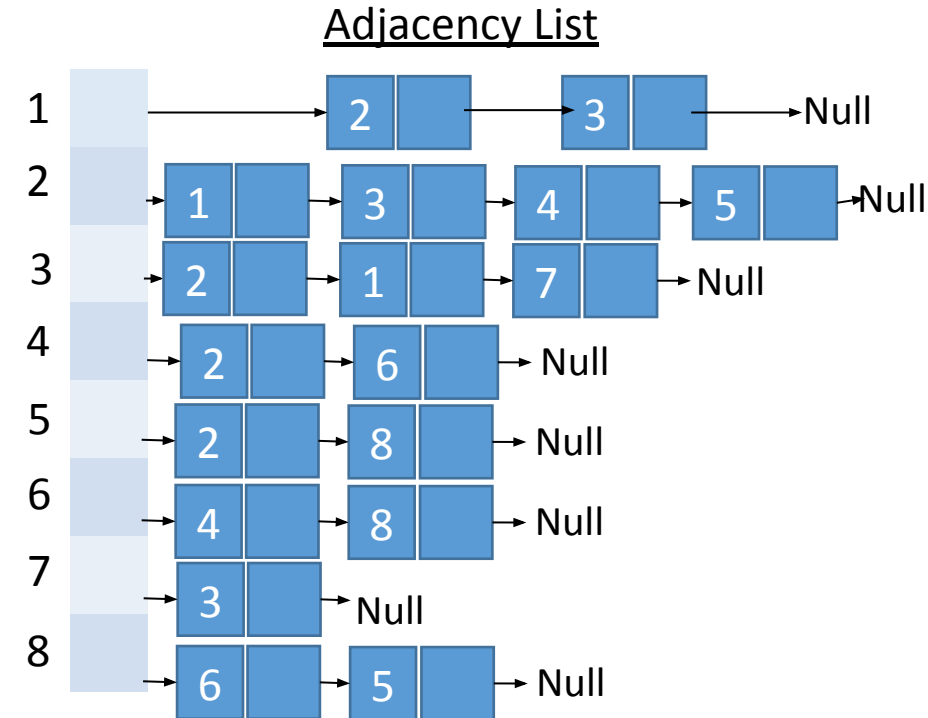
```
for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
```

```
for (int j=1;j<=8;j++){
    if(Adj[2][j]==1){
        score[j]=random();
        color[j]=random();
        parent[j]=2
    }
}
```

Let us repeat the previous task but this time by using Adjacency list

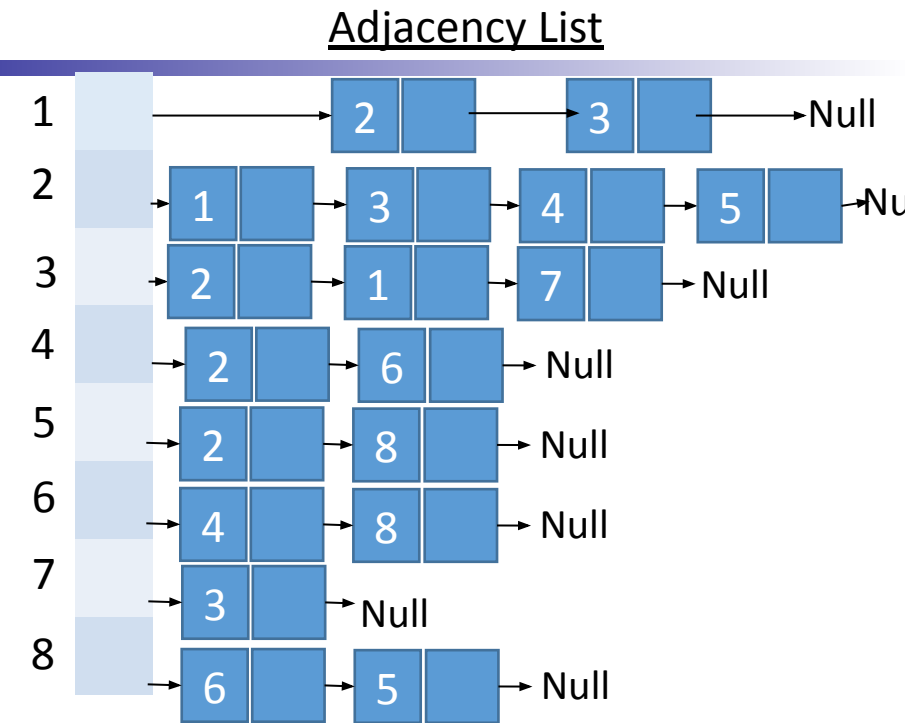
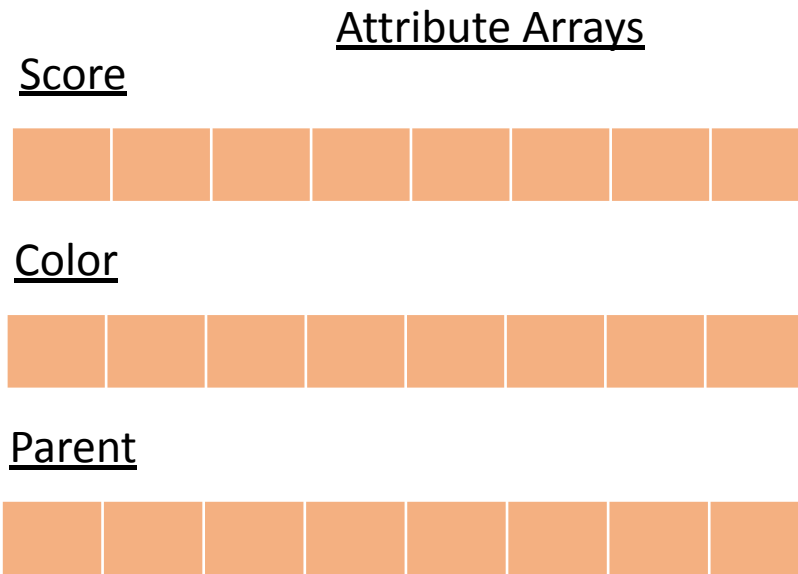
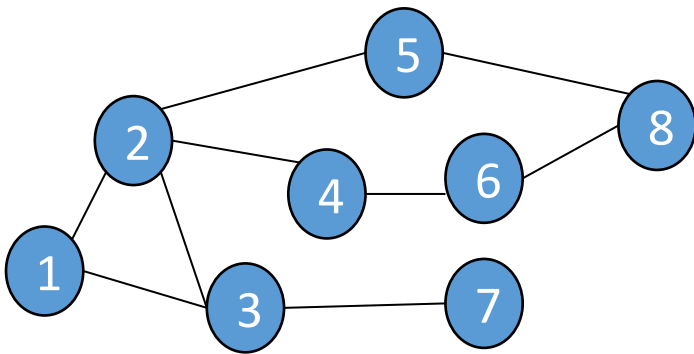


<u>Attribute Arrays</u>							
<u>Score</u>							
<u>Color</u>							
<u>Parent</u>							



```
struct adjListNode{
    int vertexNo;
    struct adjListNode* next;
}
```

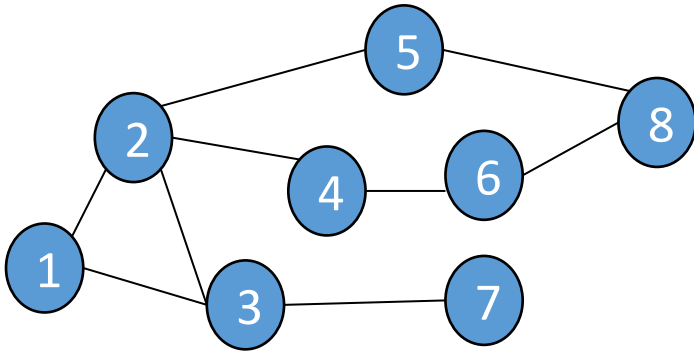
Let us repeat the previous task but this time by using Adjacency list



```

for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
  
```

Let us repeat the previous task but this time by using Adjacency list



Attribute Arrays

Score

--	--	--	--	--	--	--	--

Color

--	--	--	--	--	--	--	--

Parent

--	--	--	--	--	--	--	--

Adjacency List

1		→ 2		→ 3		→ Null
2	→ 1		→ 3		→ 4	→ 5 → Null
3	→ 2		→ 1		→ 7	→ Null
4	→ 2		→ 6		→ Null	
5	→ 2		→ 8		→ Null	
6	→ 4		→ 8		→ Null	
7	→ 3		→ Null			
8	→ 6		→ 5		→ Null	

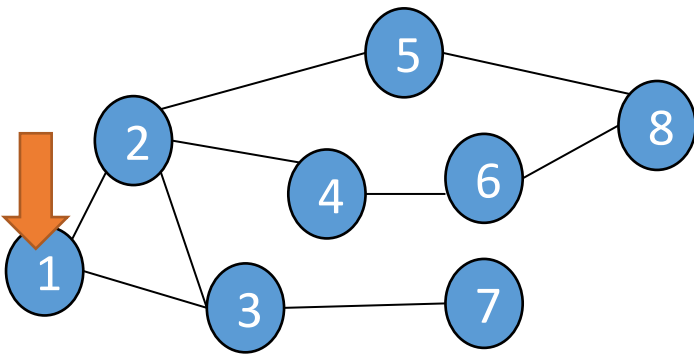
```

for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
  
```

```

traveller=adjList[2];
while traveller!=null{
    int currentVertexNo= traveller->vertexNo;
    score[currentVertexNo]=Random();
    color[currentVertexNo]=Random();
    parent[currentVertexNo]=2;
    traveller=traveller->next;
}
  
```

Let us repeat the previous task but this time by using Adjacency list



Attribute Arrays

<u>Score</u>								
<u>Color</u>								
<u>Parent</u>								

Adjacency List

Diagram illustrating an Adjacency List structure. The list shows nodes 1 through 8, each with its adjacent nodes listed in a sequence of boxes. The connections are as follows:

- Node 1: 2, 3, Null
- Node 2: 1, 3, 4, 5, Null
- Node 3: 2, 1, 7, Null
- Node 4: 2, 6, Null
- Node 5: 2, 8, Null
- Node 6: 4, 8, Null
- Node 7: 3, Null
- Node 8: 6, 5, Null

```

for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
  
```

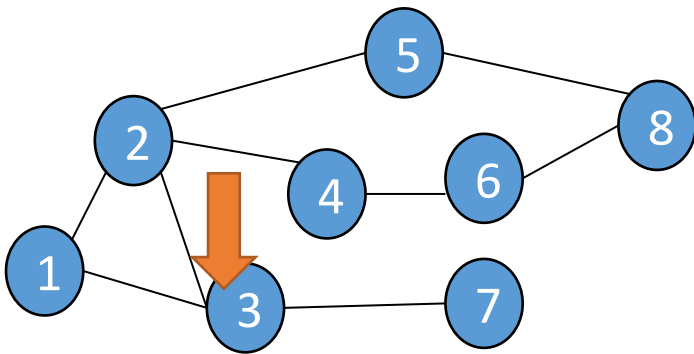
```

traveller=adjList[2];
while traveller!=null{

    traveller=traveller->next;

}
  
```

Let us repeat the previous task but this time by using Adjacency list



Attribute Arrays

<u>Score</u>								
<u>Color</u>								
<u>Parent</u>								

Adjacency List

1								
2	1		3		4		5	
3	2		1		7			
4	2		6					
5	2		8					
6	4		8					
7	3							
8	6		5					

```

traveller=adjList[2];
while traveller!=null{

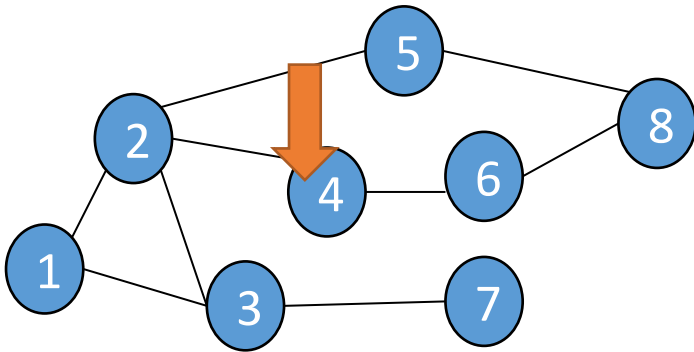
    traveller=traveller->next;

}
  
```

```

for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
  
```

Let us repeat the previous task but this time by using Adjacency list



Attribute Arrays

<u>Score</u>								
<u>Color</u>								
<u>Parent</u>								

Adjacency List

1		→ 2		→ 3		→ Null
2	→ 1		→ 3		→ 4	→ 5 → Null
3	→ 2		→ 1		→ 7	→ Null
4	→ 2		→ 6		→ Null	
5	→ 2		→ 8		→ Null	
6	→ 4		→ 8		→ Null	
7	→ 3		→ Null			
8	→ 6		→ 5		→ Null	

```

for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
  
```

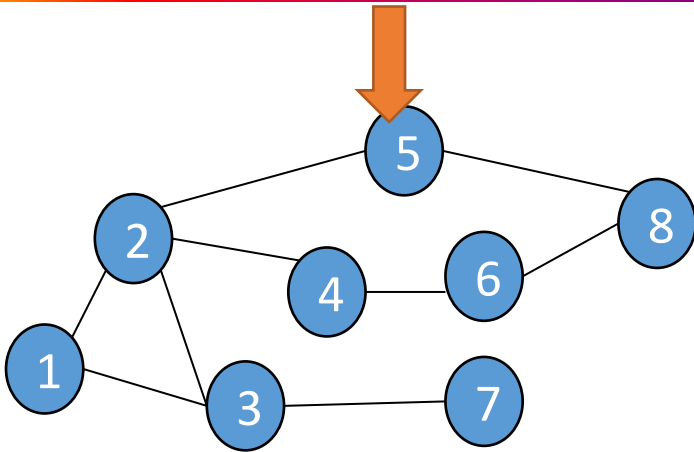
```

traveller=adjList[2];
while traveller!=null{

    traveller=traveller->next;

}
  
```


Let us repeat the previous task but this time by using Adjacency list



Attribute Arrays

<u>Score</u>								
<u>Color</u>								
<u>Parent</u>								

Adjacency List

1		→ 2		→ 3		→ Null
2	→ 1		→ 3		→ 4	→ 5 → Null
3	→ 2		→ 1		→ 7	→ Null
4	→ 2		→ 6		→ Null	
5	→ 2		→ 8		→ Null	
6	→ 4		→ 8		→ Null	
7	→ 3		→ Null			
8	→ 6		→ 5		→ Null	

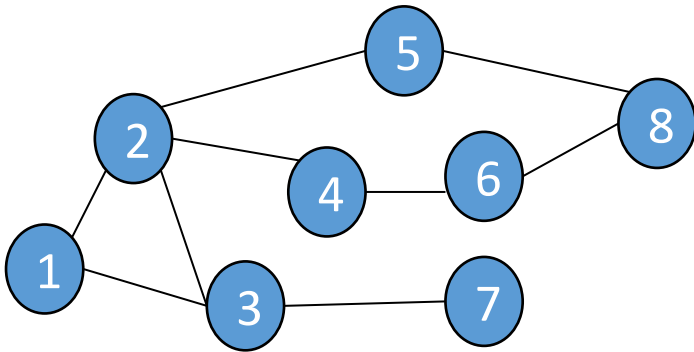
```
for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
```

```
traveller=adjList[2];
while traveller!=null{

    traveller=traveller->next;

}
```

Let us repeat the previous task but this time by using Adjacency list



Attribute Arrays

<u>Score</u>								
<u>Color</u>								
<u>Parent</u>								

Adjacency List

1		→ 2		→ 3		→ Null
2	→ 1		→ 3		→ 4	→ 5 → Null
3	→ 2		→ 1		→ 7	→ Null
4	→ 2		→ 6		→ Null	
5	→ 2		→ 8		→ Null	
6	→ 4		→ 8		→ Null	
7	→ 3		→ Null			
8	→ 6		→ 5		→ Null	

```

for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
    
```

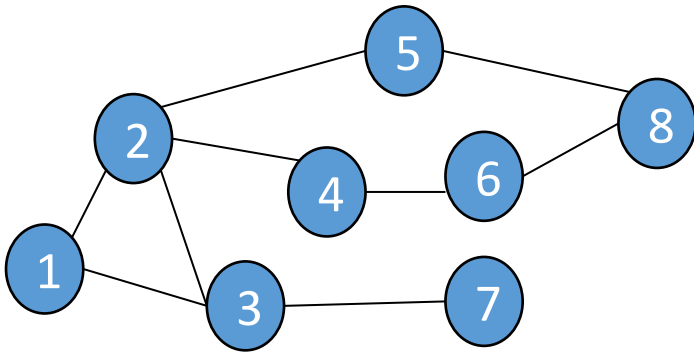
```

traveller=adjList[2];
while traveller!=null{

    traveller=traveller->next;

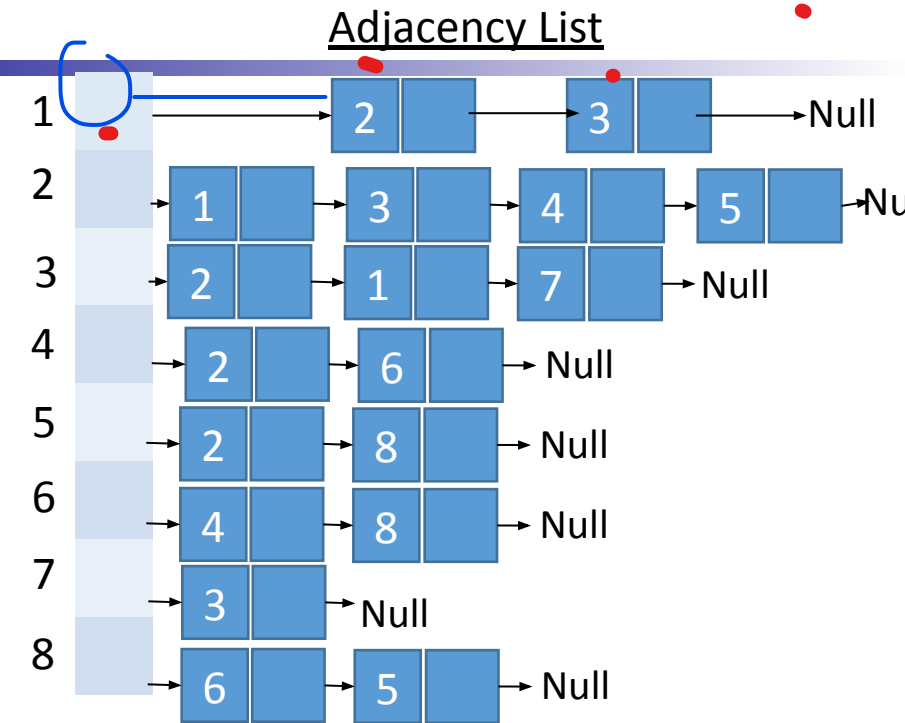
}
    
```

Let us repeat the previous task but this time by using Adjacency list



Attribute Arrays

<u>Score</u>								
<u>Color</u>								
<u>Parent</u>								



```

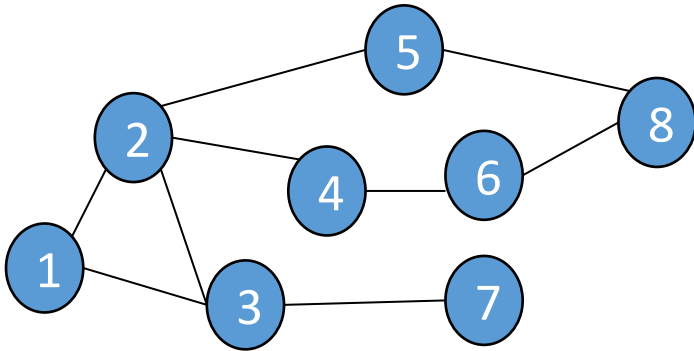
for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
    
```

```

traveller=adjList[2];
while traveller!=null{
    Score[traveller->vertexNo]=Random()

    traveller=traveller->next;
}
    
```

Let us repeat the previous task but this time by using Adjacency list



Attribute Arrays

<u>Score</u>								
<u>Color</u>								
<u>Parent</u>								

Adjacency List

1		→ 2		→ 3		→ Null
2	→ 1		→ 3		→ 4	→ 5 → Null
3	→ 2		→ 1		→ 7	→ Null
4	→ 2		→ 6		→ Null	
5	→ 2		→ 8		→ Null	
6	→ 4		→ 8		→ Null	
7	→ 3		→ Null			
8	→ 6		→ 5		→ Null	

```

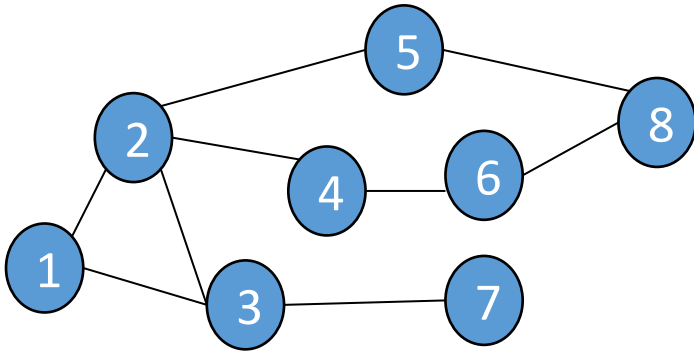
for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
  
```

```

traveller=adjList[2];
while traveller!=null{
    Score[traveller->vertexNo]=Random()
    Color[traveller->vertexNo]=Random()

    traveller=traveller->next;
}
  
```

Let us repeat the previous task but this time by using Adjacency list



Attribute Arrays

Score

--	--	--	--	--	--	--	--

Color

--	--	--	--	--	--	--	--

Parent

--	--	--	--	--	--	--	--

Adjacency List

1		→ 2		→ 3		→ Null
2	→ 1		→ 3		→ 4	→ 5 → Null
3	→ 2		→ 1		→ 7	→ Null
4	→ 2		→ 6		→ Null	
5	→ 2		→ 8		→ Null	
6	→ 4		→ 8		→ Null	
7	→ 3		→ Null			
8	→ 6		→ 5		→ Null	

```

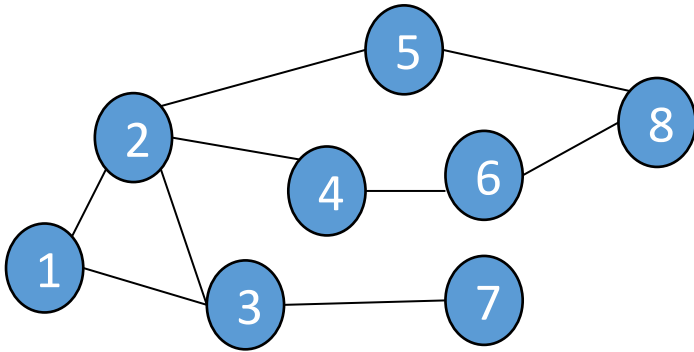
for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
    
```

```

traveller=adjList[2];
while traveller!=null{
    Score[traveller->vertexNo]=Random()
    Color[traveller->vertexNo]=Random()
    Parent[traveller->vertexNo]=2

    traveller=traveller->next;
}
    
```

Let us repeat the previous task but this time by using Adjacency list



Attribute Arrays

Score

--	--	--	--	--	--	--	--

Color

--	--	--	--	--	--	--	--

Parent

--	--	--	--	--	--	--	--

Adjacency List

1		2		3		Null
2	1	3	4	5	Null	
3	2	1	7	Null		
4	2	6	Null			
5	2	8	Null			
6	4	8	Null			
7	3	Null				
8	6	5	Null			

```

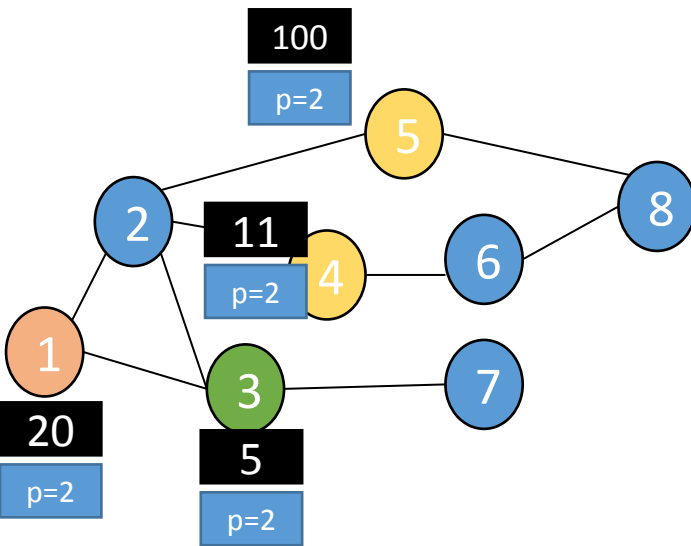
for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
    
```

```

traveller=adjList[2];
while traveller!=null{
    Score[traveller->vertexNo]=Random()
    Color[traveller->vertexNo]=Random()
    Parent[traveller->vertexNo]=2

    traveller=traveller->next;
}
    
```

Let us repeat the previous task but this time by using Adjacency list



Attribute Arrays

Score

20		5	11	100			
----	--	---	----	-----	--	--	--

Color

5		10	7	7			
---	--	----	---	---	--	--	--

Parent

2		2	2	2			
---	--	---	---	---	--	--	--

Adjacency List

1		2		3		Null
2	1	3	4	5		Null
3	2	1	7			Null
4	2	6				Null
5	2	8				Null
6	4	8				Null
7	3					Null
8	6	5				Null

```
for each vertex v adjacent to 2:
    v.score=randomNumber()
    v.color=randomColor()
    v.parent=2
```

```
traveller=adjList[2];
while traveller!=null{
    Score[traveller->vertexNo]=Random()
    Color[traveller->vertexNo]=Random()
    Parent[traveller->vertexNo]=2

    traveller=traveller->next;
}
```

Thank You

